Bayesian Exam

Contents

1. Daniel Bernolli

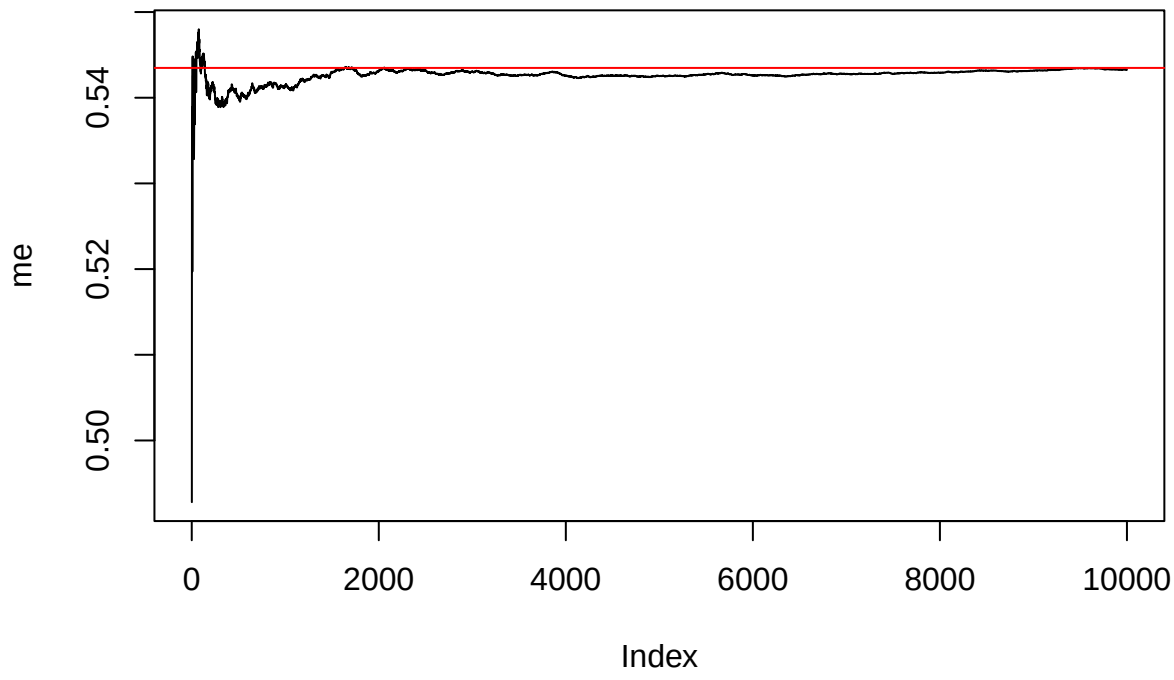
Let $y_1, \ldots, y_n \mid \theta \sim \text{Bern}(\theta)$, and assume that you have obtained a sample with $f = 35$ failures in $n = 78$ trials. Assume a $\text{Beta}(\alpha_0, \beta_0)$ prior for θ and let $\alpha_0 = \beta_0 = 7$.

(a) Convergence of Posterior Mean and Standard Deviation

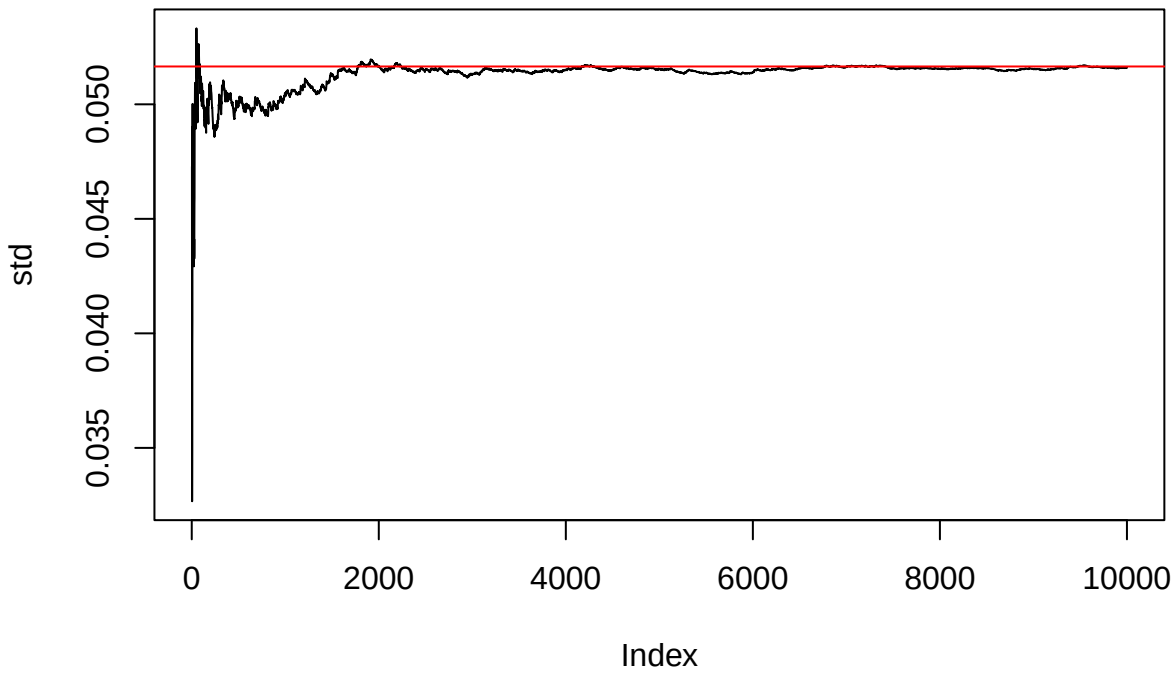
Draw 10000 random values (nDraws = 10000) from the posterior $\theta \mid y \sim \text{Beta}(\alpha_0 + s, \beta_0 + f)$, where $y = (y_1, \ldots, y_n)$, and verify graphically that the posterior mean $\mathbb{E}[\theta \mid y]$ and standard deviation $\text{SD}[\theta \mid y]$ converges to the true values as the number of random draws grows large.

Give model is a bernoulli trial and prior has a Beta(α_0, β_0) distribution, posterior $\sim$ Beta$(\alpha_0 + s, \beta_0 + f)$, where s=(n-f)=53 and f=35. Thus: posterior $\sim$ Beta$(\alpha_0 + s, \beta_0 + f)$ posterior $\sim$ Beta$(50, 42)$

```
nr <- 10000
set.seed(123456)
res <- rbeta(nr,50,42)
me <- numeric(nr)
std <- numeric(nr)

for (i in 1: nr) {
  me[i] <- mean(res[1:i])
  std[i] <- sqrt(var (res [1:i]))

}

plot(me, type = 'l')
abline(h=50/(50+42), ,col ='red')
```




```
plot(std, type = 'l')
abline(h=sqrt(50*42/(50+42)^2/(42+50+1)),col ='red')
```



(b) Posterior Probability Pr(theta > 0.5|y)

Draw 10000 random values from the posterior to compute the posterior probability $\Pr(\theta > 0.5 \mid y)$ and compare with the exact value from the Beta posterior. [Hint: use `pbeta()` to calculate the exact value].

```
sorted <- res [res>0.5]
# probability from sampling
pr <- length(sorted) / nr

# true probability
tr_pr <- 1- pbeta (0.5, 50, 42)
```

Posterior probability from sampling:

```
print(pr)
```

```
## [1] 0.8018
```

Exact value from the Beta posterior:

```
print(tr_pr)
```

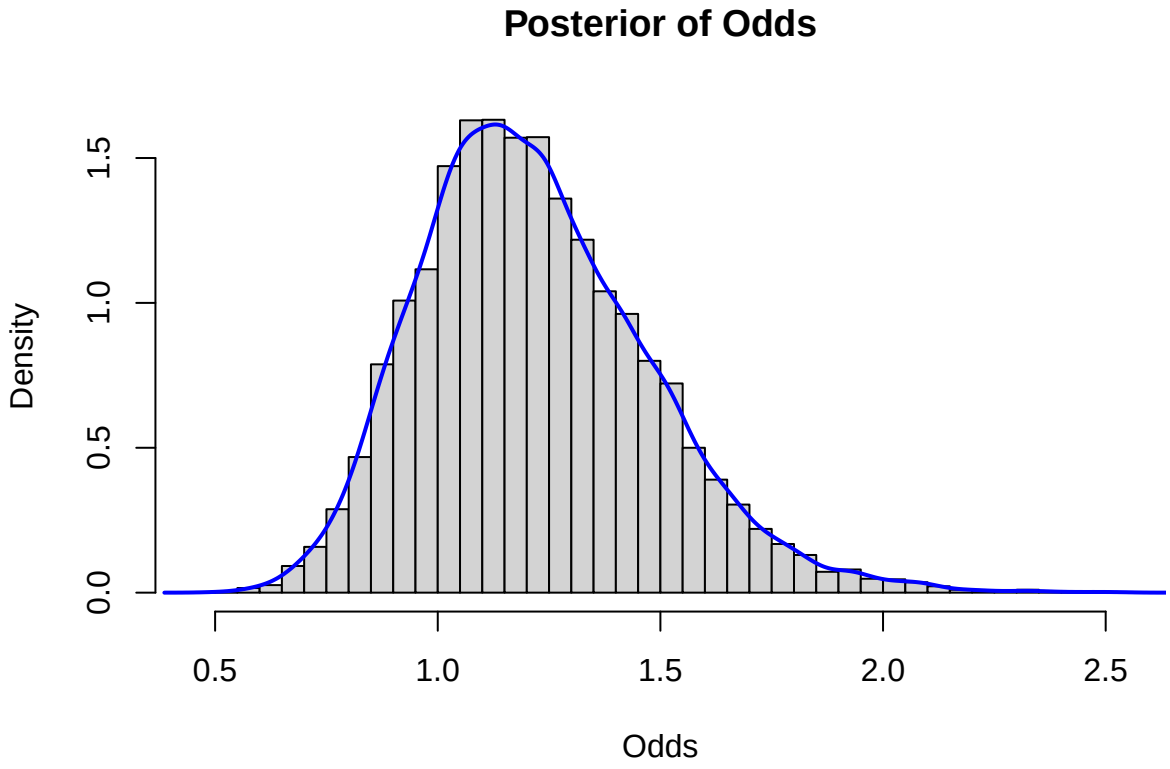
```
## [1] 0.7990936
```

The sampling gives very similar values comparing with the exact value from Beta posterior.

(c) Posterior Distribution of Odds theta = theta / (1 - theta)

Draw 10000 random values from the posterior of the odds $\phi = \frac{\theta}{1-\theta}$ by using the previous random draws from the Beta posterior for θ and plot the posterior distribution of ϕ. [Hint: `hist()` and `density()` can be utilized].

```
odds <- res / (1-res)
hist(odds, breaks = 30, freq = FALSE,probability = TRUE, main = "Posterior of Odds", xlab = "Odds")
lines(density(odds), col = "blue", lwd = 2)
```



2. Log-normal distribution and the Gini coefficient

Assume that you have asked 8 randomly selected persons about their monthly income (in thousands Swedish Krona) and obtained the following eight observations:

22, 33, 31, 49, 65, 78, 17, 24. A common model for non-negative continuous variables is the log-normal distribution. The log-normal distribution $\log N(\mu, \sigma^2)$ has density function

$$p(y \mid \mu, \sigma^2) = \frac{1}{y \cdot \sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(\log y - \mu)^2\right),$$

where $y > 0$, $-\infty < \mu < \infty$ and $\sigma^2 > 0$. The log-normal distribution is related to the normal distribution as follows: if $y \sim \log N(\mu, \sigma^2)$ then $\log y \sim N(\mu, \sigma^2)$.

Let $y_1, \ldots, y_n \mid \mu, \sigma^2 \overset{\text{iid}}{\sim} \log N(\mu, \sigma^2)$, where $\mu = 3.65$ is assumed to be known but σ^2 is unknown with non-informative prior $p(\sigma^2) \propto 1/\sigma^2$. The posterior for σ^2 is the scaled inverse chi-squared distribution, Scale-inv-$\chi^2(n, \tau^2)$, where

$$\tau^2 = \frac{\sum_{i=1}^{n} (\log y_i - \mu)^2}{n}.$$

(a) Draw 10000 random values from the posterior of sigma^2 and plot

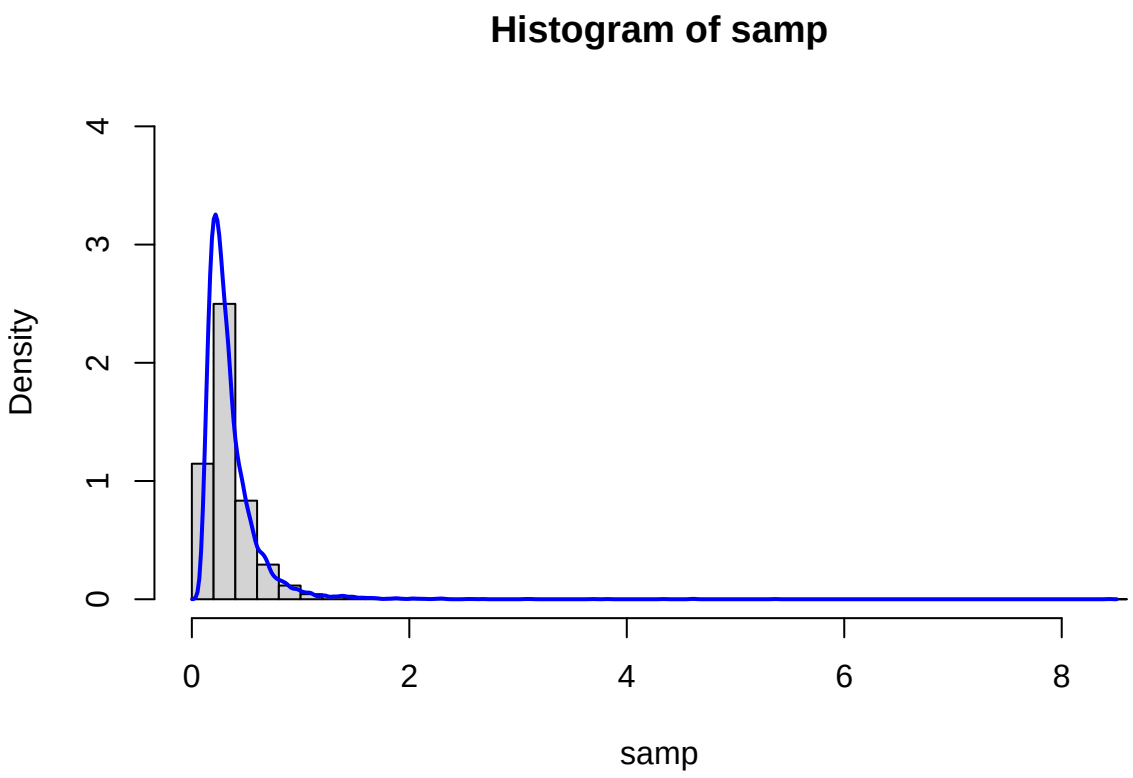
Draw 10000 random values from the posterior of σ^2 by assuming $\mu = 3.65$ and plot the posterior distribution.

Inverse-chi-squared distribution can be transferred from chi-squared distribution by the following formula:

$$\text{Inverse-}\chi^2(\text{df}, \text{scale}) = \frac{\text{df} \times \text{scale}}{\chi^2_{\text{df}}}.$$

Where `scale` (the scale parameter), and `df` (the degrees of freedom).

```
rm(list=ls())

rinvchisq <- function(n, df, scale) {
  df * scale / rchisq(n, df)
}

mu <- 3.65
arg <- c(22, 33, 31, 49, 65, 78, 17, 24)

tau_sq <- sum((log(arg) -3.65)^2)/length(arg)

set.seed(123456)
samp <- rinvchisq(10000, length(arg), tau_sq)

hist(samp, breaks = 40, freq = FALSE, ylim = c(0, 4))
lines(density(samp), col = "blue", lwd = 2)
```



? Prior Interpretation for the Posterior of σ^2

In this model, we assume that the data $y_1, \ldots, y_n$ are independent and identically distributed from a log-normal distribution:
$y_i \sim \log N(\mu, \sigma^2)$, where $\mu = 3.65$ is assumed to be known, and σ^2 is unknown.

For the prior on σ^2, we use a common non-informative prior:
$p(\sigma^2) \propto 1/\sigma^2$.
This prior is improper but commonly used in Bayesian analysis because it reflects ignorance about the scale of σ^2, and is equivalent to assuming a flat prior on $\log \sigma^2$.

This prior corresponds to a scaled inverse chi-squared distribution with prior degrees of freedom $\nu_0 = 0$, and prior scale parameter σ_0^2 undefined (since it contributes no information). Under this prior, the posterior distribution becomes:

$$\sigma^2 \mid y \sim \text{Scale-inv-}\chi^2(n, \tau^2),$$

where

$$\tau^2 = \frac{1}{n} \sum_{i=1}^{n} (\log y_i - \mu)^2.$$

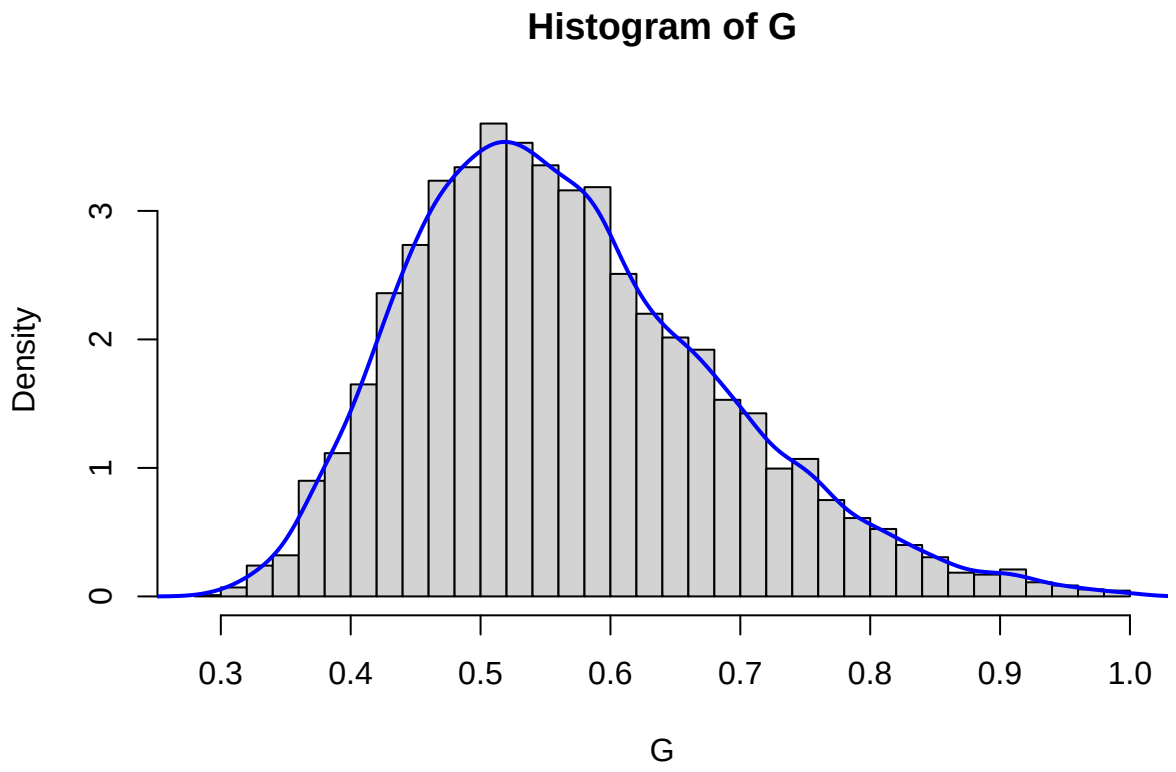
This result can be seen as a special case of the conjugate prior-posterior relationship for the normal model when the prior is non-informative. The posterior degrees of freedom are entirely determined by the number of observations n, and the posterior scale is simply the sample variance of the log-transformed data.

(b) Compute the posterior distribution of the Gini coefficient G

The most common measure of income inequality is the Gini coefficient, G, where $0 \leq G \leq 1$. $G = 0$ means a completely equal income distribution, whereas $G = 1$ means complete income inequality (see e.g. Wikipedia for more information about the Gini coefficient). It can be shown that $G = 2\Phi\left(\frac{\sigma}{\sqrt{2}}\right) - 1$ when incomes follow a $\log N(\mu, \sigma^2)$ distribution. $\Phi(z)$ is the cumulative distribution function (CDF) for the standard normal distribution with mean zero and unit variance. Use the posterior draws in a) to compute the posterior distribution of the Gini coefficient G for the current data set.

Part (a) has given information about the posterior distribution of σ^2: $\sigma^2 \mid y_1, \ldots, y_n \overset{\text{iid}}{\sim} \text{Scale-inv-}\chi^2(n, \tau^2)$, and since incomes follow a $\log N(\mu, \sigma^2)$ distribution, $G = 2\Phi\left(\frac{\sigma}{\sqrt{2}}\right) - 1$, we have

```
G <- 2* pnorm(sqrt(samp)*sqrt(2))-1
hist(G, breaks = 40, freq = FALSE)
lines(density(G), col = "blue", lwd = 2)
```



(c) Compute a 95% equal tail credible interval for G

Use the posterior draws from b) to compute a 95% equal tail credible interval for G. A 95% equal tail credible interval (a, b) cuts off 2.5% percent of the posterior probability mass to the left of a, and 2.5% to the right of b.

```
ci95 <- quantile(G, probs = c(0.025, 0.975))

ci95
```

```
##      2.5%     97.5%
## 0.3751072 0.8365435
```

(d) Compute a 95% Highest Posterior Density Interval (HPDI) for G

Use the posterior draws from b) to compute a 95% Highest Posterior Density Interval (HPDI) for G. Compare the two intervals in (c) and (d). [Hint: Use the `hdi()` function from the `bayestestR` package.]

```
library(bayestestR)
hdi(G, ci= 0.95)
```

```
## 95% HDI: [0.36, 0.81]
```

For a non symmetrical distribution where the most probability mass lies in the left side, as in this case, Highest Posterior Density Interval (HPDI) takes much probability mass from the left side comparing with qual tail credible interval. Since HPDI concentrates on part with highest probability mass, it always has a narrower interval than equal tail credible interval.

3. Bayesian inference for the rate parameter in the Poisson distribution

This exercise aims to show you that a grid approximation can be used to obtain information of the posterior distribution when the distributions for prior and posterior are not conjugate. In some cases, it is sufficient to evaluate the posterior pdf $p(\lambda \mid y)$ at a finite set of λ values to understand its structure and obtain important quantities.

The data points below represent the number of goals scored in each match of the opening week of the 2024 Swedish women's football league (Damallsvenskan):
$y = (0, 2, 5, 5, 7, 1, 4)$.
We assume that these data points are independent observations from the Poisson distribution with rate parameter $\lambda > 0$ which tells us about the average goals rate in a match. Let the prior distribution of λ be the half-normal distribution with prior pdf

$$p(\lambda \mid \sigma) = \frac{\sqrt{2}}{\sigma\sqrt{\pi}} \exp\left(-\frac{\lambda^2}{2\sigma^2}\right), \quad \lambda \geq 0,$$

and the scale parameter that is set to be $\sigma = 5$.

(a) Posterior Distribution Plot for the Average Goals Rate Parameter

Derive the expression for what the posterior pdf $p(\lambda \mid y, \sigma)$ is proportional to. Then, plot the posterior distribution of the average goals rate parameter λ over a fine grid of λ values. [Hint: you need to normalize the posterior pdf $p(\lambda \mid y, \sigma)$ so that it integrates to one.]

The posterior distribution of λ given the data y and the prior scale parameter σ is proportional to the product of the likelihood and the prior. That is,

$$p(\lambda \mid y, \sigma) \propto p(y \mid \lambda) \cdot p(\lambda \mid \sigma).$$

Since it is assumed that $y_1, y_2, \ldots, y_n \overset{\text{iid}}{\sim} \text{Poisson}(\lambda)$, then the likelihood function is given by

$$p(y \mid \lambda) = \prod_{i=1}^{n} \frac{e^{-\lambda}\lambda^{y_i}}{y_i!} \propto e^{-n\lambda}\lambda^{\sum y_i}.$$

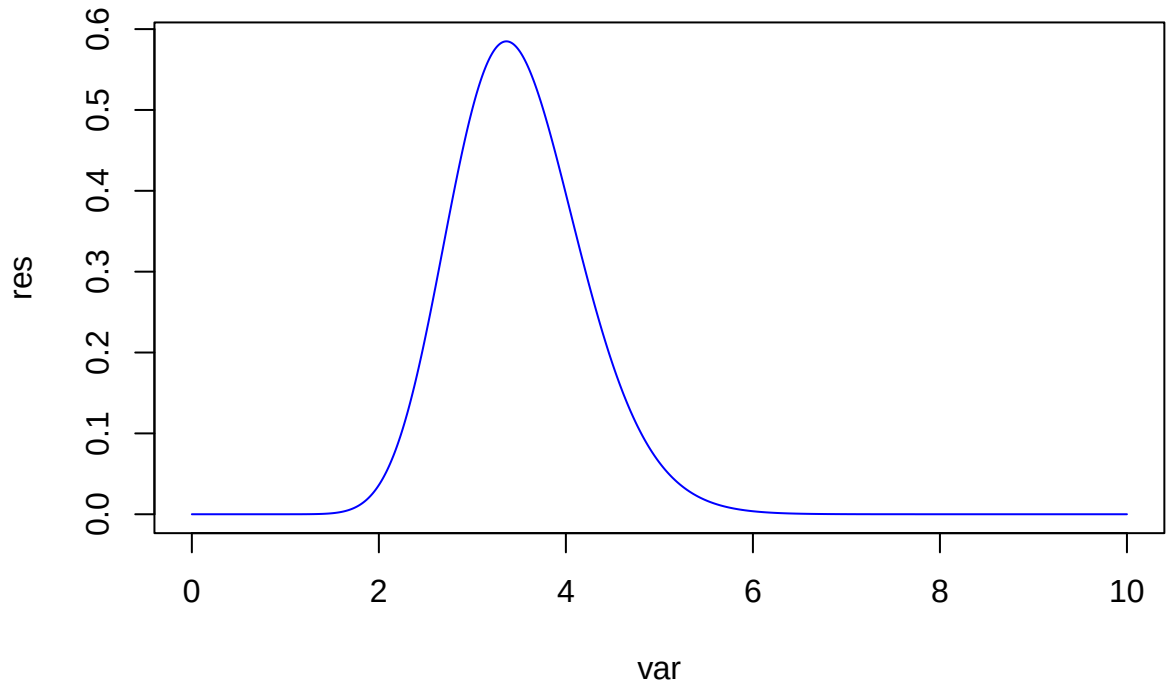
And the prior is assumed to be a Half-normal distribution:

$$p(\lambda \mid \sigma) = \frac{\sqrt{2}}{\sigma\sqrt{\pi}} \exp\left(-\frac{\lambda^2}{2\sigma^2}\right), \quad \lambda \geq 0.$$

Combining the likelihood and the prior, the posterior distribution is proportional to:

$$p(\lambda \mid y, \sigma) \propto e^{-n\lambda} \cdot \lambda^{\sum y_i} \cdot \exp\left(-\frac{\lambda^2}{2\sigma^2}\right).$$

```
rm(list= ls())
y <- c(0, 2, 5, 5, 7, 1, 4)
sig <- 5
p <- function(lambda) {

  res <- sqrt(2)/sig /sqrt(pi) * exp(-length(y)* lambda) *
    lambda^(sum(y)) * exp(-lambda^2 / 2/ sig^2)

  return(res)
}

var <- seq (0,10,0.001)

#normalize for results
res <- p(var) / sum(p(var) * 0.001)  # here have to also divide by the steps
plot(var, res, type = "l", col = "blue")
```



(b) Posterior Mode

Find the (approximate) posterior mode of λ from the information in a)

The posterior mode of λ is the value that maximizes the posterior distribution, i.e.,

$$\lambda_{\text{mode}} = \arg\max_{\lambda>0} p(\lambda \mid y, \sigma).$$

```
opt_lambda <- var[which.max(res)]
print (opt_lambda)
```

```
## [1] 3.364
```

The (approximate) posterior mode of λ is 3.364.

? Posterior Mode The posterior mode of λ is defined as the value that maximizes the posterior distribution:

$$\lambda_{\text{mode}} = \arg\max_{\lambda>0} p(\lambda \mid y, \sigma).$$

In Bayesian statistics, the posterior distribution $p(\lambda \mid y, \sigma)$ represents our updated belief about the parameter λ after observing the data y. The mode of this distribution is the most probable value — that is, the value of λ with the highest posterior density. This is also known as the maximum a posteriori (MAP) estimate.

Using the posterior mode as a point estimate is particularly useful when the posterior distribution is skewed or not symmetric, as it provides a single, most likely value for the parameter rather than relying on the mean or median, which may not be representative in such cases.

4. Linear and polynomial regression

The dataset `temp_linkoping.csv` contains daily average temperatures (in degree Celsius) in Linköping between July 2023 and June 2024. Import the dataset in R.
The response variable is `temp` and the covariate `time`, which is defined as

$$\text{time} = \frac{\text{the number of days since the beginning of the observation period}}{365}.$$

A Bayesian analysis of the following quadratic regression model is to be performed:

$$\text{temp} = \beta_0 + \beta_1 \cdot \text{time} + \beta_2 \cdot \text{time}^2 + \varepsilon, \quad \varepsilon \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

(a) Prior Specification and Prior Predictive Check for Quadratic Model Use the conjugate prior for the linear regression model. The prior hyperparameters μ_0, Ω_0, ν_0 and σ_0^2 shall be set to sensible values. Start with

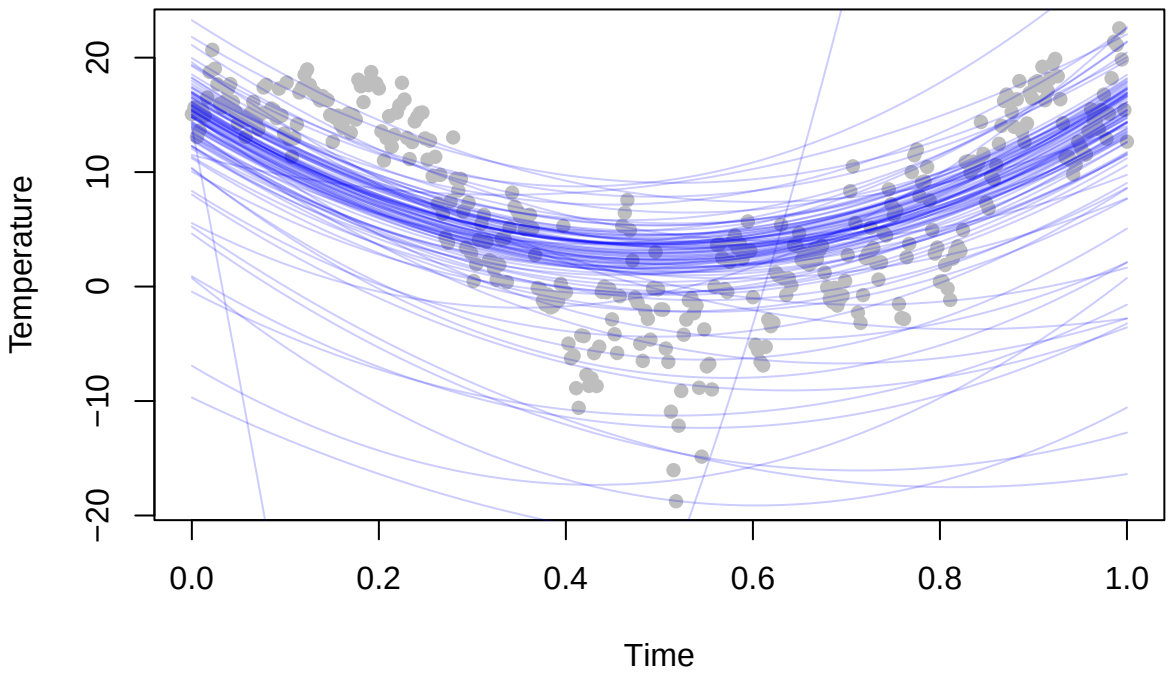
$$\mu_0 = (0, 100, -100)^T, \quad \Omega_0 = 0.01 \cdot I_3, \quad \nu_0 = 1, \quad \sigma_0^2 = 1.$$

Check if this prior agrees with your prior opinions by simulating draws from the joint prior of all parameters and for every draw compute the regression curve.
This gives a collection of regression curves; one for each draw from the prior.
Does the collection of curves look reasonable? If not, change the prior hyperparameters until the collection of prior regression curves agrees with your prior beliefs about the regression curve.
Hint: R package ***mvtnorm*** *can be used and your Scaled-Inv-*χ^2 simulator of random draws from Lab 1.

```
data <- read.csv('temp_linkoping.csv')
library (mvtnorm)
# define inverse chi square function
#self-defined prior hyperparameters
mu_0 <- c(15, -50, 50)
nu_0<- 1
sig_0_sq <- 2
omega_0 <- diag(3)*0.8


 rinvchisq <- function(n, df, scale) {
  df * scale / rchisq(n, df)
  }
```

Join prior for bete given

```
# sort our time and time^2
data['time^2'] <- data[3]^2
data['time^0'] <- data[3]^0
time <- as.matrix(data[, c(6,3,5)])
temp <- as.matrix(data[, 4])

curve <-  matrix(NA, nrow = 366, ncol = 100)

for  (i in 1: 100) {
  # sample for sigma square first
  sig_sq_sp<- rinvchisq(1,nu_0, sig_0_sq)

  # beta from sampeled sigma square
  samp <- rmvnorm(n=1, mean= mu_0, sigma = sig_sq_sp*solve(omega_0))
  pred  <- time %*% as.numeric(samp)
  curve [,i] <- pred
}

plot(time[,2], temp, pch = 16, col = 'gray', xlab = "Time", ylab = "Temperature")
for (i in 2:100) {
  lines(data[,3], curve[,i], col = rgb(0,0,1,alpha=0.2))
}
```



? Bayesian Linear Regression with Conjugate Prior We consider a Bayesian linear regression model with a conjugate prior. The response variable (e.g., temperature) is modeled as a quadratic function of time:

$$f(\text{time}) = \mathbb{E}[\text{temp}|\text{time}] = \beta_0 + \beta_1 \cdot \text{time} + \beta_2 \cdot \text{time}^2,$$

The model assumes a Gaussian likelihood and conjugate priors:

- The prior for
$$\beta \mid \sigma^2$$
is:
$$\beta \mid \sigma^2 \sim \mathcal{N}(\mu_0, \sigma^2 W_0^{-1})$$
- The prior for the variance is an inverse-chi-squared distribution:
$$\sigma^2 \sim \text{Inv-}\chi^2(\nu_0, \sigma_0^2)$$

Given observed data, the posterior distributions are:

- For
$$\beta \mid \sigma^2, \mathrm{y}$$
:

$$\beta \mid \sigma^2, \mathrm{y} \sim \mathcal{N}(\mu_n, \sigma^2 W_n^{-1})$$

- For
$$\sigma^2 \mid \mathrm{y}$$
:

$$\sigma^2 \mid \mathrm{y} \sim \text{Inv-}\chi^2(\nu_n, \sigma_0^2)$$

The updated posterior parameters are computed as:

$$W_n = W_0 + X^\top X$$

$$\mu_n = W_n^{-1}(W_0\mu_0 + X^\top y)$$

with $y = X\beta + \varepsilon$, the standard linear regression form. In calculation just use $y = X\beta$.

$$\nu_n = \nu_0 + n$$

$$\nu_n\sigma_0^2 = \nu_0\sigma_0^2 + (\mu_0^\top W_0\mu_0 + y^\top y - \mu_n^\top W_n\mu_n)$$

This conjugate setup ensures the posterior remains in the same family as the prior, allowing for analytic updates and efficient inference.

(b) Function to Simulate from Joint Posterior Distribution Write a function that simulates draws from the joint posterior distribution of β_0, β_1, β_2 and σ^2.

i. Plot a histogram for each marginal posterior of the parameters.

ii. Make a scatter plot of the temperature data and overlay a curve for the posterior median of the regression function

$$f(\text{time}) = \mathbb{E}[\text{temp}|\text{time}] = \beta_0 + \beta_1 \cdot \text{time} + \beta_2 \cdot \text{time}^2,$$

i.e., the median of $f(\text{time})$ is computed for every value of `time`.
In addition, overlay curves for the 90% equal tail posterior probability intervals of $f(\text{time})$, i.e., the 5 and 95 posterior percentiles of $f(\text{time})$ are computed for every value of `time`.
Does the posterior probability interval contain most of the data points? Should they?

Part 1 Plot a histogram for each marginal posterior of the parameters Since we use a Normal-Inverse-Chi-Squared prior on (β, σ^2), where:

- $\beta \mid \sigma^2 \sim \mathcal{N}(\mu_0, \sigma^2 \Omega_0^{-1})$
- $\sigma^2 \sim \text{Inv-}\chi^2(\nu_0, s_0)$

Given observed data (X, y), the joint posterior distribution of β and σ^2 is: - The marginal posterior of σ^2 is:

$$\sigma^2 \mid y \sim \text{Inv-}\chi^2(\nu_n, \sigma_n)$$

with:

$$\nu_n = \nu_0 + n$$

and

$$\nu_n \sigma_n^2 = \nu_0 \sigma_0^2 + y^\top y + \mu_0^\top \Omega_0 \mu_0 - \mu_n^\top \Omega_n \mu_n$$

where the updated parameters are:

$$\Omega_n = \Omega_0 + X^\top X$$

$$\mu_n = \Omega_n^{-1}(\Omega_0 \mu_0 + X^\top y)$$

- The conditional posterior of β given σ^2 and data is:

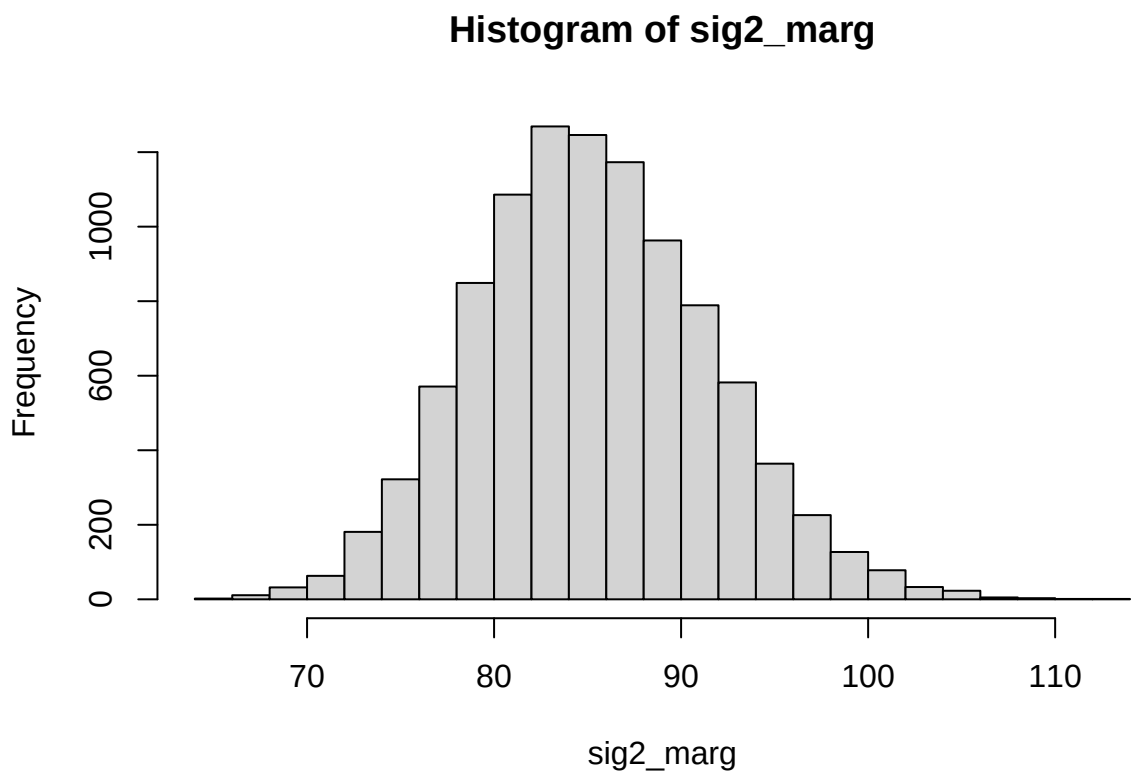
$$\beta \mid \sigma^2, y \sim \mathcal{N}(\mu_n, \sigma^2 \Omega_n^{-1})$$

```
set.seed(123456)
sample_posterior <- function (number ,nu_0, sig_0_sq, omega_0) {
  # The marginal posterior of sigma

nu_n <- nu_0+366 ## n=366, 366 observations
omega_n <- t(time) %*%time+omega_0
mu_n <- solve(omega_n) %*% (omega_0%*%mu_0 +t(time)%*%(temp))
sig_n_sq <- as.numeric(1/nu_n*(nu_0*sig_0_sq+
                                 t(temp)%*%temp+
                                 t(mu_0)%*%omega_n%*%mu_0-
                                 t(mu_n)%*%omega_n%*%mu_n))

# sampled results
# joint posterior of sigma square given data
  sig2_marg<- rinvchisq(number,nu_n, sig_n_sq)

# joint posterior of beta given sigma square and data
beta_marg <- matrix(nrow=number, ncol=3)
colnames(beta_marg) = c('beta0', 'beta1','beta2')

 for (i in 1: number) {
     beta_marg[i,] <- as.vector(rmvnorm(n=1, mean= mu_n,
                                        sigma = sig2_marg[i]*solve(omega_n)))
 }
```

```
return(list(sig2_marg = sig2_marg, beta_marg = beta_marg))
}

res <-sample_posterior(number=10000 ,nu_0=nu_0, sig_0_sq=sig_0_sq, omega_0=omega_0)
sig2_marg <- res$sig2_marg
beta_marg <- res$beta_marg
```

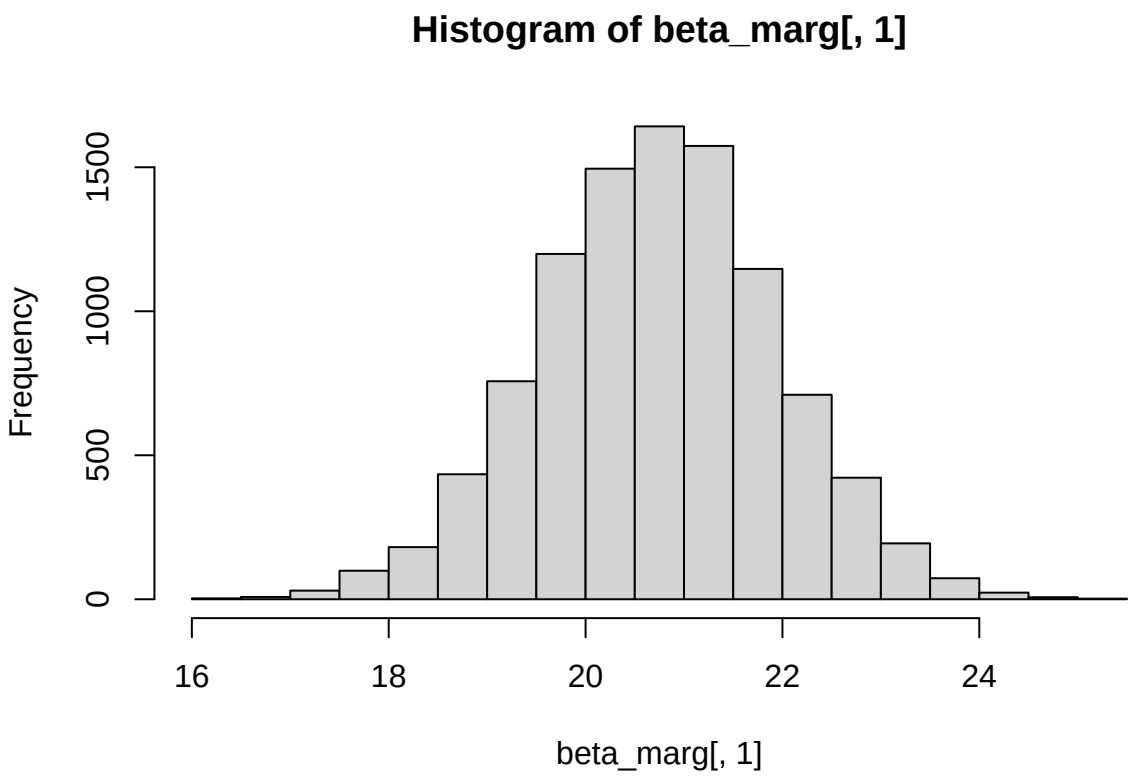
Then we can plot the histogram for σ^2 given data:

```
hist(sig2_marg, breaks = 20)
```



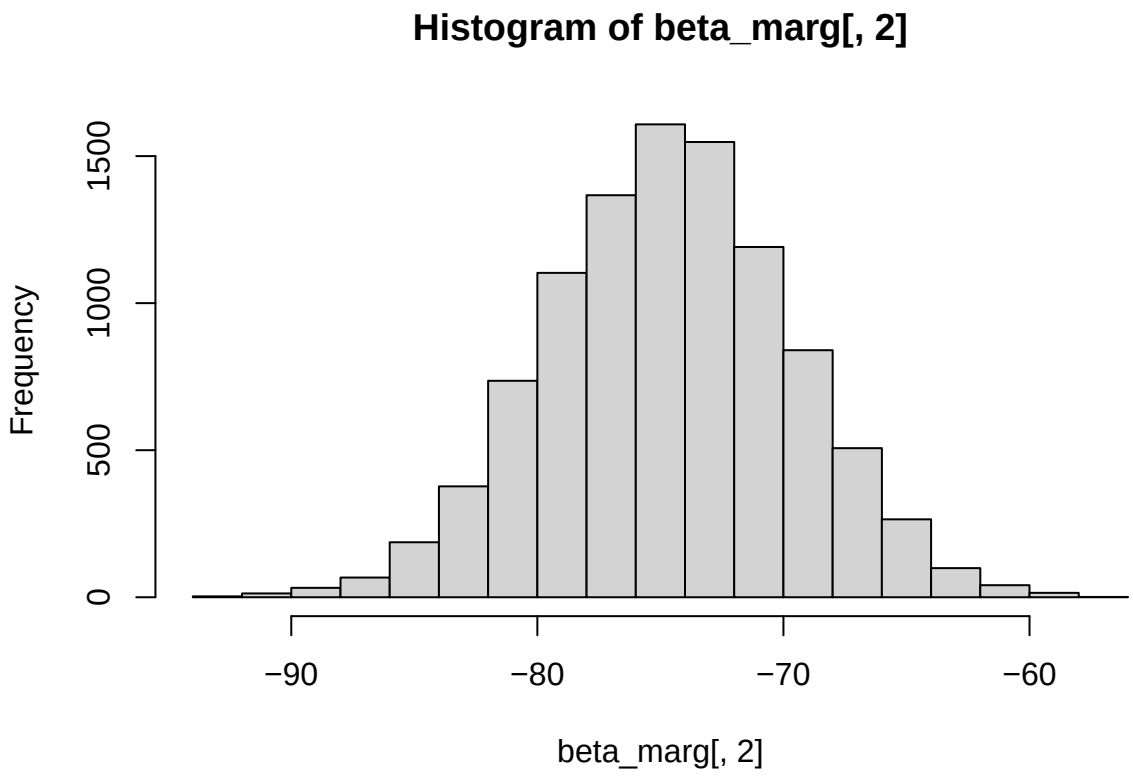
The histogram for β_0 given σ^2 and data:

```
hist(beta_marg[,1], breaks = 20)
```



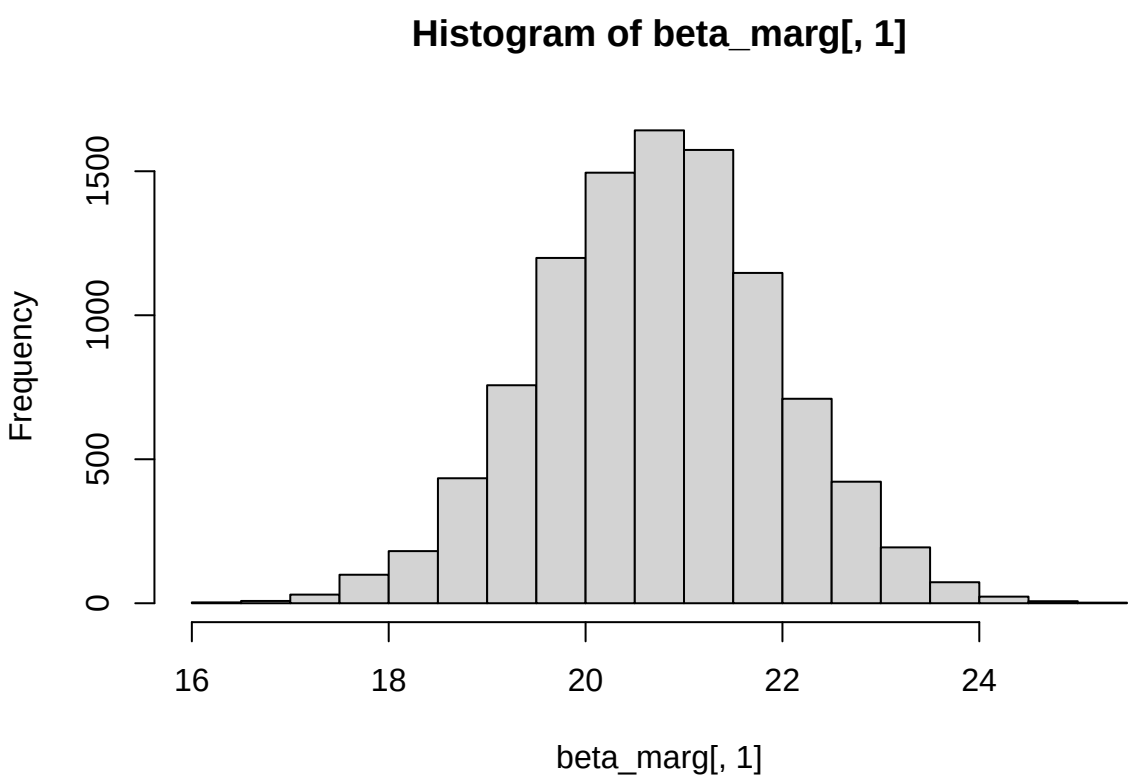
The histogram for β_1 given σ^2 and data:

```
hist(beta_marg[,2], breaks = 20)
```



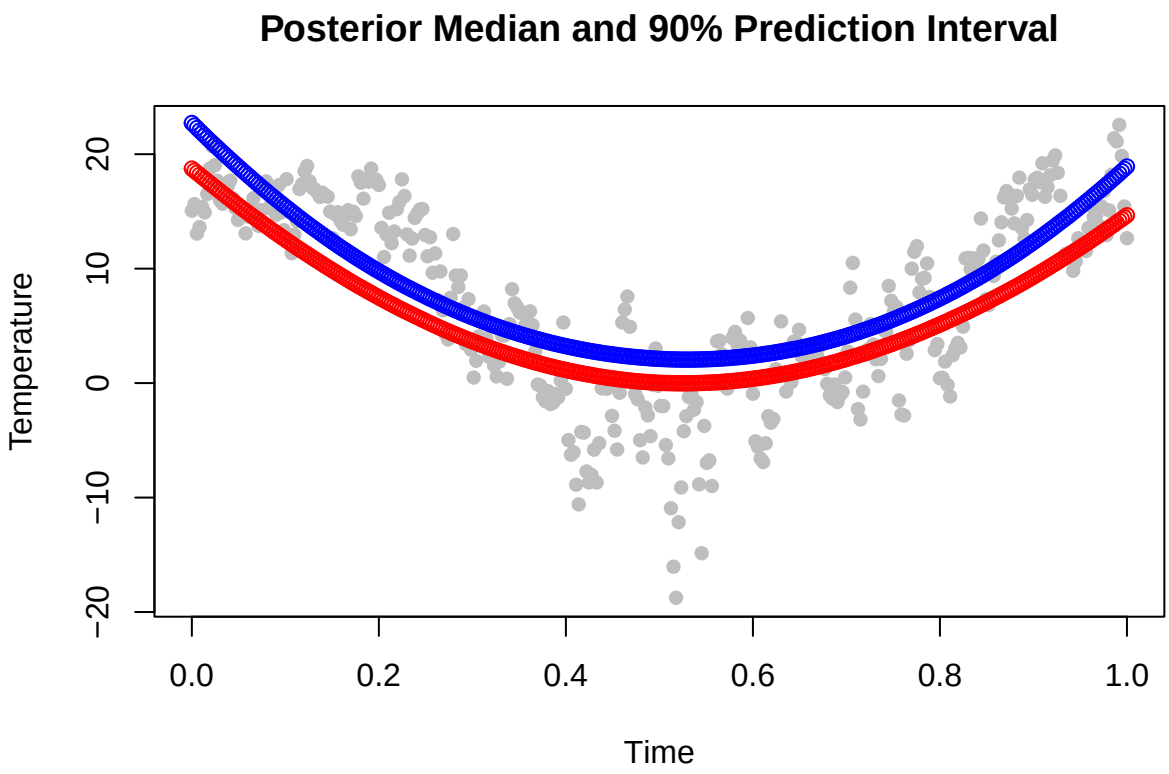
The histogram for β_2 given σ^2 and data:

```
hist(beta_marg[,1], breaks = 20)
```




```
pred_temp <- time%*%t(beta_marg)
pred_temp_meadian <- apply(pred_temp,1, quantile, probs = c(0.05,0.95) )

plot(time[,2], temp, pch = 16, col = 'gray', xlab = "Time", ylab = "Temperature",
     main = "Posterior Median and 90% Prediction Interval")
points(time[,2], pred_temp_meadian[1,], col ='red')
points(time[,2], pred_temp_meadian[2,], col ='blue')
```

Part 2 Make a scatter plot of the temperature data and overlay posterior curves



(c) Posterior Distribution of $\tilde{x}$ (time with lowest expected temperature) It is of interest to locate the time with the lowest expected temperature (i.e., the time where $f(\text{time})$ is minimal). Let's call this value $\tilde{x}$.
Use the simulated draws in (b) to simulate from the posterior distribution of $\tilde{x}$.
You can solve this analytically or numerically.
Hint: The regression curve is a quadratic polynomial. Given each posterior draw of β_0, β_1, and β_2, you can find a simple formula for $\tilde{x}$.

To find the time point $\tilde{x}$ at which the function reaches its minimum , we compute the derivative with respect to time:

$$\frac{d}{d\,\text{time}} f(\text{time}) = \beta_1 + 2\beta_2 \cdot \text{time}$$

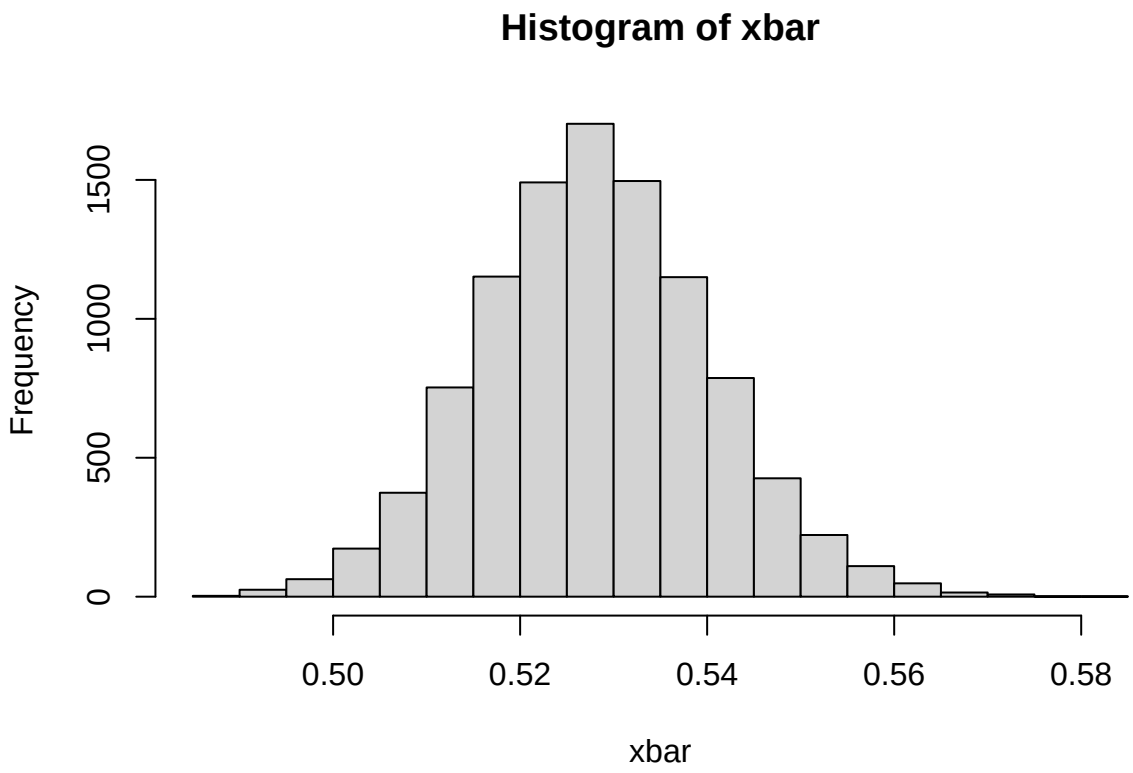
Setting the derivative equal to zero:

$$\beta_1 + 2\beta_2 \cdot \tilde{x} = 0$$

Solving for $\tilde{x}$ gives:

$$\tilde{x} = -\frac{\beta_1}{2\beta_2}$$

```r
xbar <- -beta_marg[,2]/2/beta_marg[,3]
hist(xbar, breaks = 20)
```



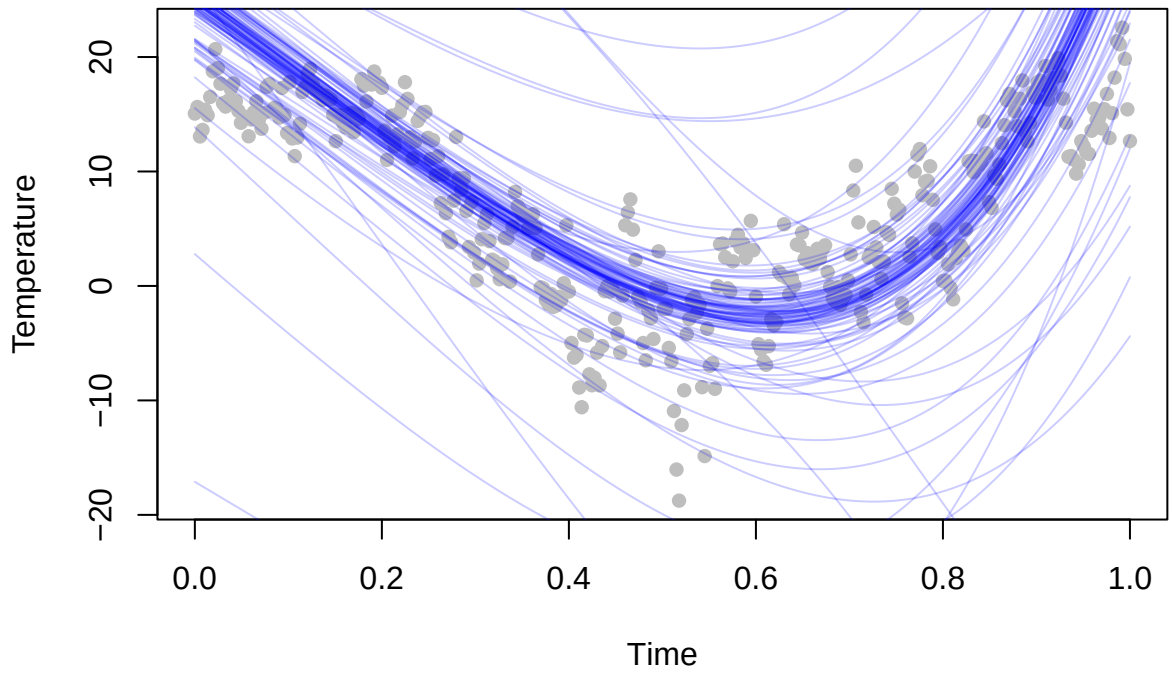
(d) Higher Order Polynomial Regression (Order 10) with Regularizing Prior Say now that you want to estimate a polynomial regression of order 10, but you suspect that higher order terms may not be needed, and you worry about overfitting the data.
Suggest a suitable prior that mitigates this potential problem and motivate your choice.
Repeat task (a) for the selected prior and the higher order polynomial to see if it behaves as expected.
Hint: The task is to specify μ_0 and Ω_0 in a suitable way.

To reflect our prior belief that lower-order terms are likely important, we assign relatively low precision values (small diagonal values (1) in Ω) to them.On the other hand, for higher-order terms (e.g., degree 5 to 10), we set much higher prior precision (1000), encouraging these coefficients to be small unless the data provides strong evidence otherwise.

It is shown that the curve fit the actual data better than what we have in task (a), but no significant improvement is obtained than what we have from task (b) where we have even lower order. Therefore, we believe that higher order terms are not needed.

```r
# define inverse chi square function
mu_0_new <- c(25, -60, 0.01,-1,60,10,0,0,0,0,0)
nu_0<- 1
sig_0_sq <- 2
omega_0_new <- diag(c(rep(1, 6), rep(1000, 5)))
```

```r
#construct poly terms
x <- data[, 3]
poly_terms <- sapply(0:10, function(p) x^p)
colnames(poly_terms) <- paste0("time^", 0:10)
time_new <- as.matrix(poly_terms)

# sort our time and time^2

curve_new <-  matrix(NA, nrow = 366, ncol = 100)

for  (i in 1: 100) {
  sig_sq_sp<- rinvchisq(1,nu_0, sig_0_sq)
  samp_new <- rmvnorm(n=1, mean= mu_0_new, sigma = sig_sq_sp*solve(omega_0_new))
  pred_new  <- time_new %*% as.numeric(samp_new)
  curve_new [,i] <- pred_new
}

plot(time[,2], temp, pch = 16, col = 'gray', xlab = "Time",
     ylab = "Temperature"#,ylim=range(curve_new)
     )
for (i in 2:100) {
  lines(data[,3], curve_new[,i], col = rgb(0,0,1,alpha=0.2))
}
```



5. Posterior approximation for classification with logistic regression

The dataset Disease.xlsx contains n = 313 observations on the following seven variables related to a certain disease:

Variable	Data type	Meaning	Role
Class_of_diagnosis	Binary	Disease or not?	Response y
Gender	Binary	Woman (1) or Male (0)	Feature
Age	Counts	Age	Feature
Diagnosis_method	Binary	1 for a certain diagnosis method	Feature
Duration_of_symptoms	Numeric	Duration of symptoms in days	Feature
Dyspnoea	Binary	1 if person has laboured breathing	Feature
White_blood	Counts	N white blood cells per microliter	Feature

?Derivation of the Log-Posterior in Logistic Regression We begin by specifying the probability model used in logistic regression. The likelihood for each observation is given by the logistic (sigmoid) function:

$$P(y_i = 1 \mid x_i, \beta) = \frac{1}{1 + \exp(-x_i^\top \beta)}.$$

The Bernoulli likelihood for a single observation

$$y_i \in \{0, 1\}$$

is:

$$p(y_i \mid x_i, \beta) = \left(\frac{1}{1 + \exp(-x_i^\top \beta)}\right)^{y_i} \left(1 - \frac{1}{1 + \exp(-x_i^\top \beta)}\right)^{1-y_i}.$$

Simplifying the second term using:

$$1 - \frac{1}{1 + \exp(-z)} = \frac{\exp(-z)}{1 + \exp(-z)},$$

we get the complete likelihood for a single observation:

$$p(y_i \mid x_i, \beta) = \left(\frac{\exp(x_i^\top \beta)}{1 + \exp(x_i^\top \beta)}\right)^{y_i} \left(\frac{1}{1 + \exp(x_i^\top \beta)}\right)^{1-y_i}.$$

The likelihood for the full dataset with n observations is:

$$p(y \mid \beta, X) = \prod_{i=1}^{n} \left(\frac{\exp(x_i^\top \beta)}{1 + \exp(x_i^\top \beta)}\right)^{y_i} \left(\frac{1}{1 + \exp(x_i^\top \beta)}\right)^{1-y_i}.$$

Taking the logarithm of the likelihood gives the log-likelihood:

$$\log p(y \mid \beta, X) = \sum_{i=1}^{n} \left[y_i (x_i^\top \beta) - \log(1 + \exp(x_i^\top \beta)) \right].$$

This is the log-likelihood function used in the optimization. In R code, it is typically written as:

```
lin_pred <- X %*% beta
log_lik <- sum(y * lin_pred - log(1 + exp(lin_pred)))
```

Note: In some implementations, to improve numerical stability, the log-sum-exp trick is used to avoid overflow in `log(1 + exp(x))`.

Finally, if a prior distribution is placed on

$$\beta$$

, such as a Gaussian prior

$$\beta \sim \mathcal{N}(0, \tau^2 I)$$

, then the **log-posterior** becomes:

$$\log p(\beta \mid y, X) \propto \log p(y \mid \beta, X) - \frac{1}{2\tau^2} \beta^\top \beta.$$

This is the target function optimized numerically to obtain the posterior mode

$$\tilde{\beta}$$

.

?Bayesian Logistic Regression: Log-Posterior Formulation In Bayesian logistic regression, the probability of a binary outcome is modeled using the logistic function:

$$P(y_i = 1 \mid x_i, \beta) = \frac{1}{1 + \exp(-x_i^\top \beta)}.$$

Given observed outcomes

$$y_i \in \{0, 1\}$$

, the log-likelihood for all

$$n$$

observations is:

$$\log p(y \mid \beta, X) = \sum_{i=1}^{n} \left[y_i (x_i^\top \beta) - \log(1 + \exp(x_i^\top \beta)) \right].$$

NOTICE: log_lik <- sum(y * lin_pred - log1p(exp(lin_pred))), 其中 log1p(x) 是 $\log(1+x)$ 的稳定实现，用于防止数值溢出。

By Bayes' theorem, the posterior is proportional to the product of the likelihood and the prior. Taking logs gives the log-posterior (up to a constant):

$$\log p(\beta \mid y, X) \propto \log p(y \mid \beta, X) + \log p(\beta).$$

The posterior distribution of

$$\beta$$

is typically not available in closed form for logistic regression due to the non-conjugate likelihood. However, we can approximate the posterior using a second-order Taylor expansion of the log-posterior around its mode.

Starting from the log-posterior expression:

$$\log p(\beta \mid y, X) \propto \log p(y \mid \beta, X) + \log p(\beta),$$

we define

$$\tilde{\beta}$$

as the value of

$$\beta$$

that maximizes this log-posterior, i.e., the posterior mode.

Using a Laplace approximation, we approximate the posterior distribution with a Gaussian centered at the mode

$$\tilde{\beta}$$

and covariance given by the inverse of the negative Hessian of the log-posterior evaluated at

$$\tilde{\beta}$$

. This gives:

$$\beta \mid y, X \approx \mathcal{N}(\tilde{\beta}, J_y^{-1}(\tilde{\beta})),$$

where

$$J_y(\tilde{\beta})$$

is the observed information matrix:

$$J_y(\tilde{\beta}) = -\nabla^2 \log p(\beta \mid y, X)\big|_{\beta=\tilde{\beta}}.$$

This approximation is useful for inference and posterior summaries, especially when exact sampling is computationally expensive.

(a) Posterior Approximation of Logistic Regression Parameters ( ) Consider the logistic regression model:

$$\Pr(y = 1|x, \beta) = \frac{\exp(x^T\beta)}{1 + \exp(x^T\beta)},$$

where y equals 1 if the person has a disease and 0 if not. x is a 7-dimensional vector containing six features and a column of 1's to include an intercept in the model. The goal is to approximate the posterior distribution of the parameter vector with a multivariate normal distribution

$$\beta|y, x \sim \mathcal{N}(\tilde{\beta}, J_y^{-1}(\tilde{\beta})),$$

where $\tilde{\beta}$ is the posterior mode and $J(\tilde{\beta}) = -\frac{\partial^2 \ln p(\beta|y)}{\partial\beta\partial\beta^T}\Big|_{\beta=\tilde{\beta}}$ is the negative of the observed Hessian evaluated at the posterior mode. Note that $\frac{\partial^2 \ln p(\beta|y)}{\partial\beta\partial\beta^T}$ is a 7×7 matrix with second derivatives on the diagonal and cross-derivatives $\frac{\partial^2 \ln p(\beta|y)}{\partial\beta_i\partial\beta_j}$ on the off-diagonal. You can compute this derivative by hand, but we will let the computer do it numerically for you. Calculate both $\tilde{\beta}$ and $J(\tilde{\beta})$ by using the `optim` function in R.

Hint: You may use code snippets from my demo of logistic regression in Lecture 6. Use the prior $\beta \sim \mathcal{N}(0, \tau^2 I)$, where $\tau = 2$.
Present the numerical values of $\tilde{\beta}$ and $J_y^{-1}(\tilde{\beta})$ for the Disease data. Compute an approximate 95% equal tail posterior probability interval for the regression coefficient to the variable Age. Would you say that this feature is of importance for the probability that a person has the disease?

Hint: You can verify that your estimation results are reasonable by comparing the posterior means to the maximum likelihood estimates, given by: glmModel <- glm(Class_of_diagnosis ~ 0 + ., data = Disease, family = binomial).]

```
library(mvtnorm)

data <- read.csv("Disease.csv")
Nobs <- dim(data)[1] # number of observations
#data <- data.frame(intercept=rep(1,Nobs),disease_data) # add intercept
data[,c("age","duration_of_symptoms","white_blood" )] <-
scale(data[,c("age","duration_of_symptoms","white_blood" )])

Xnames = colnames(data)[1:6]
```

```
LogPostLogistic <- function(beta,y,X,tau){
  lin_pred <- X %*% beta
  p = length(beta)
  log_lik <- sum(y * lin_pred - log1p(exp(lin_pred)))
  #log_prior <- -sum(beta^2) / (2 * tau^2)
  log_prior = dmvnorm(beta, rep(0, p), tau^2 * diag(p), log=TRUE);
  return(log_lik + log_prior)
}
```

```
lambda <- 1#
tau=2
Npar = dim(data)[2] - 1
mu <- as.matrix(rep(0,Npar)) #
initVal <- matrix(0,Npar,1)
Sigma <- (1/lambda)*diag(Npar)
y =  as.numeric(data$class_of_diagnosis)
X = as.matrix(data[,1:ncol(data) - 1])# Select which covariates/features to include

OptimRes <- optim(par = rep(0, ncol(X)),
                  fn = LogPostLogistic,
                  X = X, y = y, tau = tau,
                  method = "BFGS",
                  control = list(fnscale = -1, maxit = 1000),
                  hessian = TRUE)

approxPostMode <- matrix(OptimRes$par,1,6)
cat("The posterior mode:",OptimRes$par) # The posterior mode
```

```
## The posterior mode: -0.3882422 -0.2661479 -0.6423896 0.19313 -0.1375241 -0.04498936
```

```
cov_matrix <- solve(-OptimRes$hessian)  # J^{-1}
approxPostStd <- sqrt(diag(cov_matrix)) # `Computing approximate standard deviations.
```

```r
Cred_int <- matrix(0,2,6)
# Create 95 % approximate credibility intervals for each coefficient
Cred_int[1,] <- approxPostMode - 1.96*approxPostStd
Cred_int[2,] <- approxPostMode + 1.96*approxPostStd

colnames(Cred_int) <- Xnames
rownames(Cred_int) <- c("LCI","UCI")
#LCI（Lower Confidence Interval）UCI（Upper Confidence Interval）
print(Cred_int)
```

```
##      Intercept         age     gender duration_of_symptoms   dyspnoea
## LCI -1.0010275 -0.51707150 -1.1350628          -0.04685009 -0.7496778
## UCI  0.2245431 -0.01522428 -0.1497163           0.43311005  0.4746297
##     white_blood
## LCI  -0.2954176
## UCI   0.2054389
```

```r
# 使用 glm 验证
glm_data <- data.frame(
  Class_of_diagnosis = data$class_of_diagnosis,
  Gender = data$gender,
  Age_z = data$age,
  Duration_z = data$duration_of_symptoms,
  Dyspnoea = data$dyspnoea,
  White_z = data$white_blood
)

glm_model <- glm(Class_of_diagnosis ~ Gender + Age_z + Duration_z + Dyspnoea + White_z,
                 data = glm_data, family = binomial)
print(summary(glm_model))
```

```
##
## Call:
## glm(formula = Class_of_diagnosis ~ Gender + Age_z + Duration_z +
##     Dyspnoea + White_z, family = binomial, data = glm_data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.38961    0.31999  -1.218   0.2234
## Gender      -0.64938    0.25415  -2.555   0.0106 *
## Age_z       -0.26762    0.12839  -2.084   0.0371 *
## Duration_z   0.19367    0.12270   1.578   0.1145
## Dyspnoea    -0.13348    0.31924  -0.418   0.6759
## White_z     -0.04504    0.12813  -0.352   0.7252
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 384.25  on 312  degrees of freedom
## Residual deviance: 369.86  on 307  degrees of freedom
## AIC: 381.86
```

```
##
## Number of Fisher Scoring iterations: 4
```

```
cat("\n95% CI for Age coefficient: [", Cred_int[1,2], ", ", Cred_int[2,2], "]\n")
```

```
##
## 95% CI for Age coefficient: [ -0.5170715 ,  -0.01522428 ]
```

```
cat("MAP of age coefficient: ", approxPostMode[2], "\n")
```

```
## MAP of age coefficient:  -0.2661479
```

```
cat("Posterior mean of age coefficient: ", OptimRes$par[2], "\n")
```

```
## Posterior mean of age coefficient:  -0.2661479
```

```
cat("glm model coefficient of age: ", coef(glm_model)[2], "\n")
```

```
## glm model coefficient of age:  -0.6493847
```

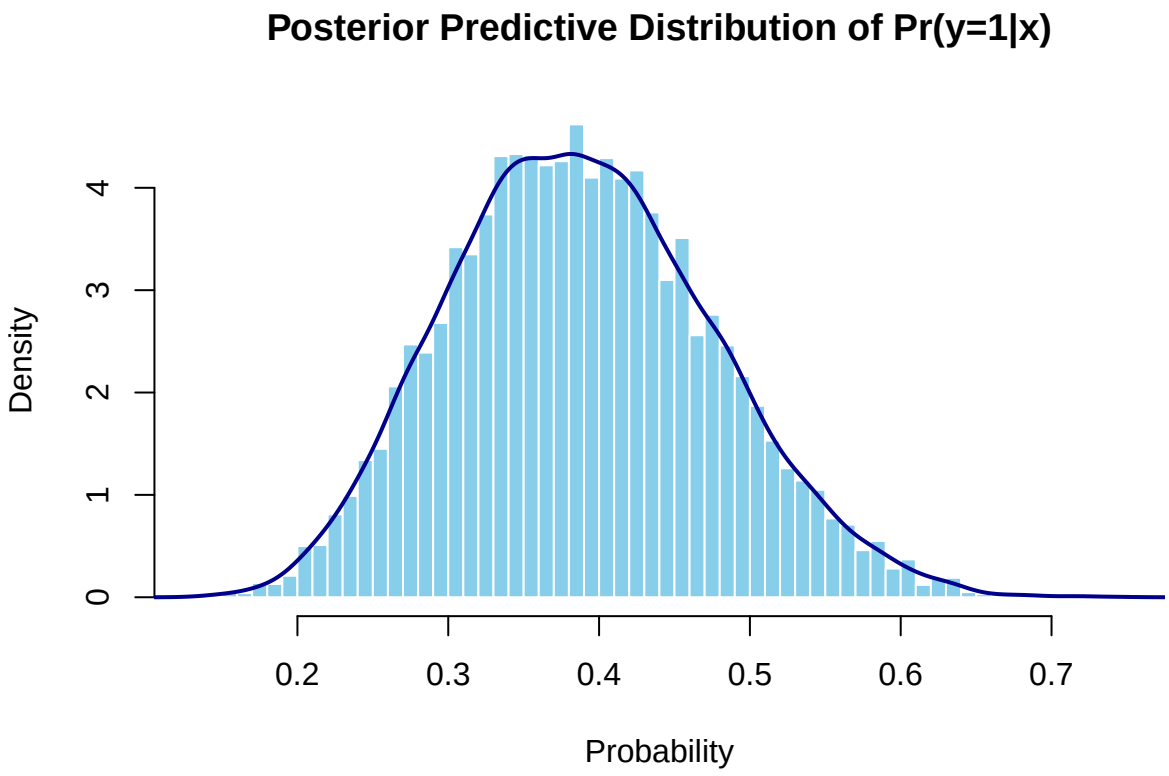
```
#Report the posterior mean (Estimate) and 95% confidence interval (LCI, UCI),
 # Age:  -0.26762 (95% CI: -0.015, -0.51)
```

The 95% confidence interval of the Age variable does not include 0, result is [-0.51, -0.015] ,MAP is -0.27, so it has a significant impact on the probability of illness.

(b) Posterior Predictive Distribution of Pr(y=1 x) Use your normal approximation to the posterior from (a). Write a function that simulate draws from the posterior predictive distribution of $\Pr(y = 1|x)$, where the certain diagnosis method was used and where the values of x corresponds to a 38-year-old woman, with 10 days of symptoms, no laboured breathing and 11000 white blood cells per microliter. Plot the posterior predictive distribution of $\Pr(y = 1|x)$ for this person.

Hints: The R package ***mvtnorm*** *will be useful. Remember that* $\Pr(y = 1|x)$ can be calculated for each posterior draw of .

```
set.seed(123)
data_b <- read.csv("Disease.csv")

# standardize the numeric features
# model was trained on standardized variables.To make predictions, the new data
#must be on the same scale.
new_age <- (38 - mean(data_b$age)) / sd(data_b$age)
new_duration <- (10 - mean(data_b$duration_of_symptoms)) / sd(data_b$duration_of_symptoms)
new_white <- (11000 - mean(data_b$white_blood)) / sd(data_b$white_blood)
X_new <- c(1, new_age, 1, new_duration, 0, new_white)

samples <- rmvnorm(10000, mean = OptimRes$par, sigma = cov_matrix)
lin_pred <- samples %*% X_new
pr <- exp(lin_pred) / (1 + exp(lin_pred))

hist(pr, breaks = 50,prob = TRUE,main = "Posterior Predictive Distribution of Pr(y=1|x)",
     xlab = "Probability", col = "skyblue", border = "white",)
lines(density(pr), col = "darkblue", lwd = 2)
```




```
#When you plot a histogram with prob = TRUE, or when you use density(), the
#y-axis represents density, not probability. That means the height of the curve
#can exceed 1, as long as the total area under the curve is 1.
```

? Gibbs Sampling for Bayesian Logistic Regression using Polya-Gamma Augmentation In Bayesian logistic regression, direct sampling from the posterior distribution of the coefficients β is challenging due to the non-conjugate likelihood. One efficient approach to address this is **Polya-Gamma data augmentation**, which introduces latent variables to make the conditional distributions tractable for Gibbs sampling.

We assume a prior for the regression coefficients:

$$\beta \sim \mathcal{N}(b, B),$$

and introduce latent variables $w = (w_1, \dots, w_n)$ such that the joint posterior becomes:

$$p(\beta, w \mid y) \propto p(y \mid \beta, w)\, p(w \mid \beta)\, p(\beta).$$

In each iteration of the Gibbs sampler:

1. For $i = 1, \dots, n$, sample each latent variable:

$$w_i \mid \beta \sim \mathrm{PG}(1, x_i^\top \beta),$$

where $\mathrm{PG}(1, \cdot)$ denotes the Polya-Gamma distribution.

2. Then sample the regression coefficients from the conditional Gaussian posterior:

$$\beta \mid y, w \sim \mathcal{N}(m_w, V_w),$$

where

$$V_w = \left(X^\top W X + B^{-1}\right)^{-1}, \quad m_w = V_w \left(X^\top \kappa + B^{-1} b\right),$$

with $W = \text{diag}(w_1, \ldots, w_n)$ and $\kappa_i = y_i - \frac{1}{2}$.

This augmentation makes each full conditional distribution easy to sample from, enabling efficient posterior inference through Gibbs sampling.

?Inefficiency Factor (IF) In Markov Chain Monte Carlo (MCMC), the samples $\{q^{(1)}, q^{(2)}, \ldots, q^{(N)}\}$ are usually autocorrelated, which means that the variance of the sample mean $\bar{q}$ does not decrease as fast as it would with independent samples.

The **variance of the sample mean** $\bar{q}$ can be expressed as:

$$\text{Var}(\bar{q}) = \frac{\sigma^2}{N} \cdot \text{IF},$$

where σ^2 is the marginal variance of $q^{(i)}$, and IF is the **inefficiency factor**, also called the **integrated autocorrelation time**.

The inefficiency factor is defined as:

$$\text{IF} = 1 + 2 \sum_{k=1}^{\infty} \rho_k,$$

where ρ_k is the lag-k autocorrelation of the MCMC samples. This expression shows how autocorrelation inflates the variance of the estimator. When samples are independent, all $\rho_k = 0$, and IF $= 1$.

In practice, the infinite sum is truncated at some lag K where ρ_k becomes negligible, giving the estimate:

$$\widehat{\text{IF}} = 1 + 2 \sum_{k=1}^{K} \hat{\rho}_k.$$

A lower IF indicates more efficient sampling, with IF close to 1 being ideal.

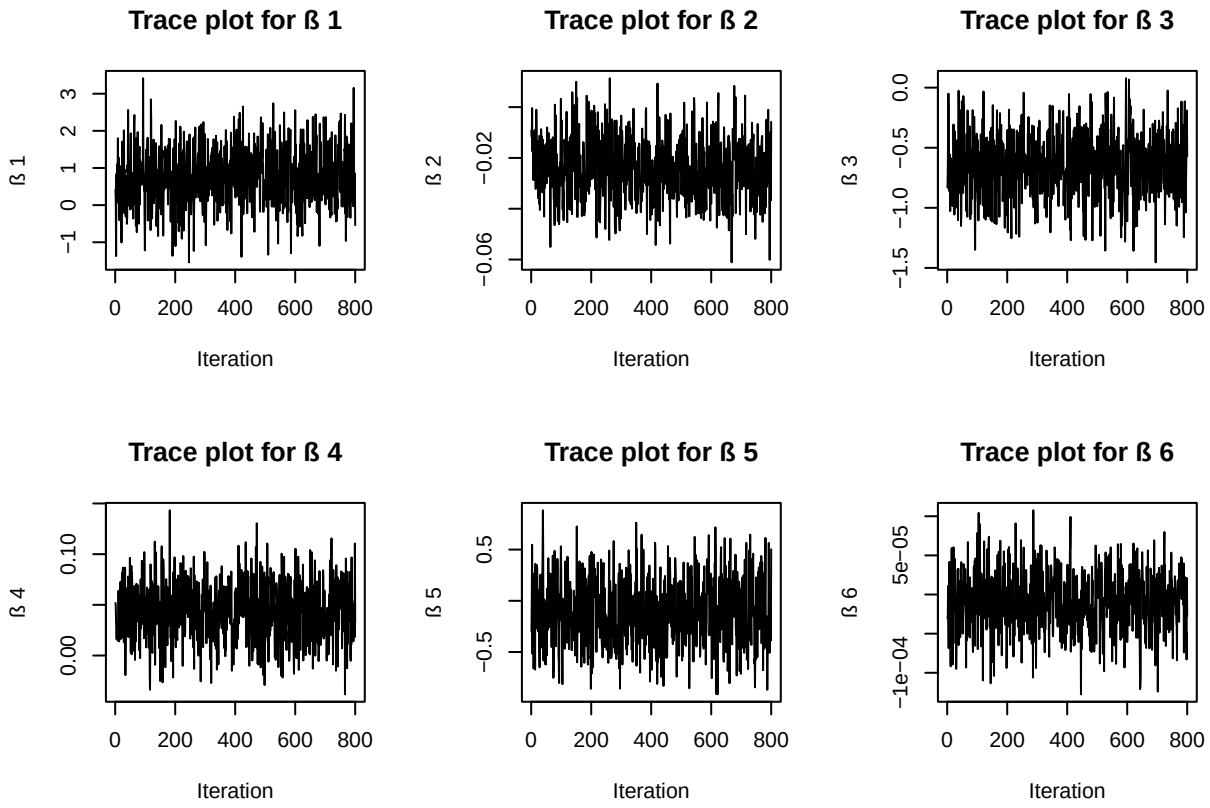
6. Gibbs sampling for the logistic regression

Consider again the logistic regression model in problem 2 from the previous computer lab 2. Use the prior $\beta \sim \mathcal{N}(0, \tau^2 I)$, where $\tau = 3$.

(a) Gibbs Sampler Implementation for Logistic Regression with Polya-Gamma Augmentation

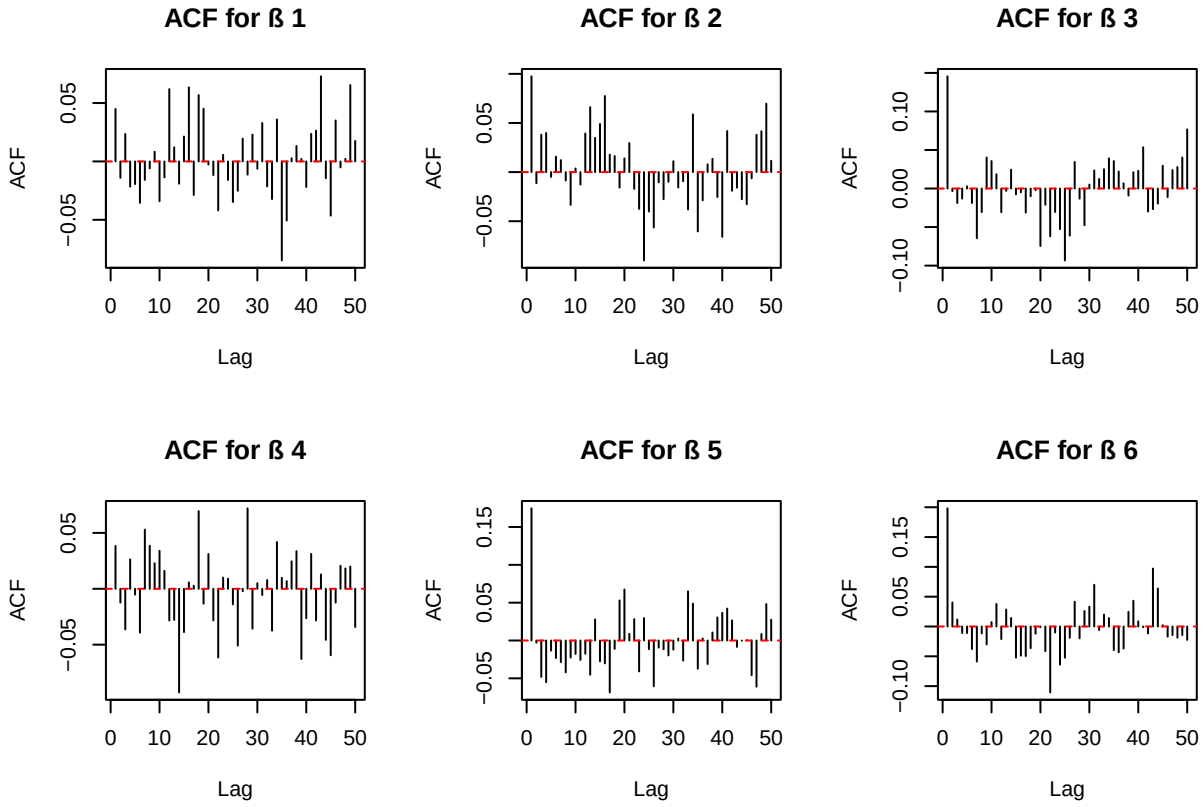
Implement (code!) a Gibbs sampler that simulates from the joint posterior $p(\omega, \beta|x)$ by augmenting the data with Polya-gamma latent variables $\omega_i, i = 1, \ldots, n$. The full conditional posteriors are given on the slides from Lecture 7. Evaluate the convergence of the Gibbs sampler by calculating the Inefficiency Factors (IFs) and by plotting the trajectories of the sampled Markov chains.

```r
library(BayesLogit)
library(mvtnorm)

data = read.csv('Disease.csv')
x <- as.matrix(data [, 1:6])
y <- as.matrix(data [, 7])
# initialize beta
nDraws = 1000
burnin <- 200       # Burn-in period

n <- nrow(x)
p <- ncol(x)

beta <- rep(1, 6) # 5+1 intercept

beta_samples <- matrix(NA, nDraws, p)

set.seed(123)

for (i in 1:nDraws) {
  # sample for wi|beta
  omega <- rpg(n, 1, x %*% beta)

  k <- y - 1 / 2
  B <- 9 * diag(6)
  b <- rep(0, p)
  # Conditional posterior for beta

  v_omg <- solve(t(x) %*% diag(omega) %*% x + solve(B))
  m_omg <- v_omg %*% (t(x) %*% k + solve(B) %*% b)

  beta <- as.numeric(rmvnorm(1, mean = m_omg, sigma = v_omg))
  beta_samples[i, ] <- beta
}
# Discard burn-in
beta_samples <- beta_samples[(burnin + 1):nDraws, ]
nPost <- nrow(beta_samples)

par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    beta_samples[, i],
    type = 'l',
    main = paste("Trace plot for  ", i),
    xlab = "Iteration",
    ylab = paste(" ", i)
  )
}
```




```r
max_lag <- 50  # set the max lag
acfs <- matrix(NA, max_lag, p)

for (i in 1:p) {
  chain <- beta_samples[, i] - mean(beta_samples[, i])
  var_chain <- mean(chain ^ 2)
  for (j in 1:max_lag) {  # j as lag
    # calculate the autocorrelation factor
    acfs[j, i] <- sum(chain[1:(nPost - j)] * chain[(j + 1):nPost]) / nPost / var_chain

  }

}

par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    1:max_lag,
    acfs[, i],
    type = 'h',
    main = paste("ACF for  ", i),
    xlab = "Lag",
    ylab = "ACF"
  )
  abline(h = 0, col = "red", lty = 2)
```

```
}
```




```
# inefficiency factor (IF)
IFs <- numeric(p)

for (i in 1:p) {
  IFs[i] <- 1 + 2 * sum(acfs[, i])
  cat(paste("IF for  ", i, "=", round(IFs[i], 2), "\n"))
}
```

```
## IF for   1 = 1.16
## IF for   2 = 1.25
## IF for   3 = 0.95
## IF for   4 = 0.72
## IF for   5 = 0.77
## IF for   6 = 0.72
```

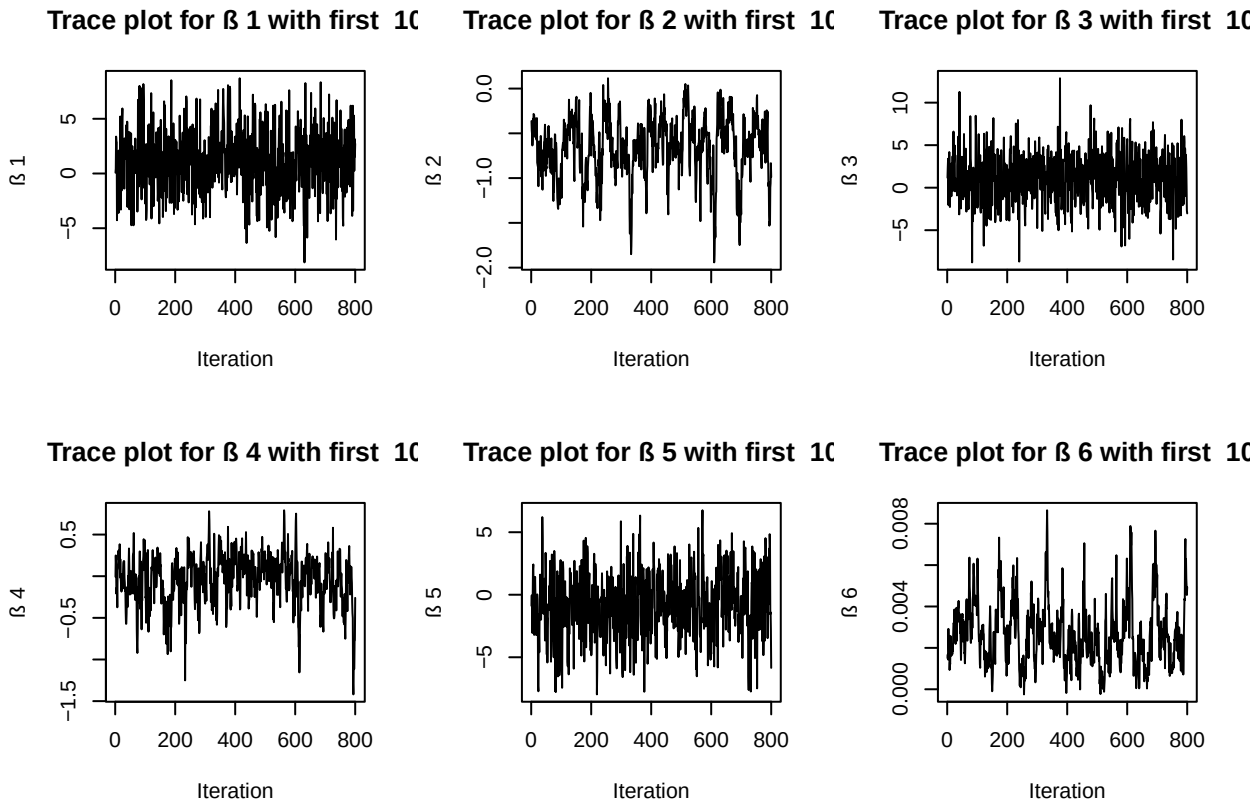
Plots show that all efficient fluctuate around certain levels without any obvious tendency or any stuck. And calculated inefficiency factors are around 1, which means very sampling are performed with quite low dependency and the efficiency is very good.

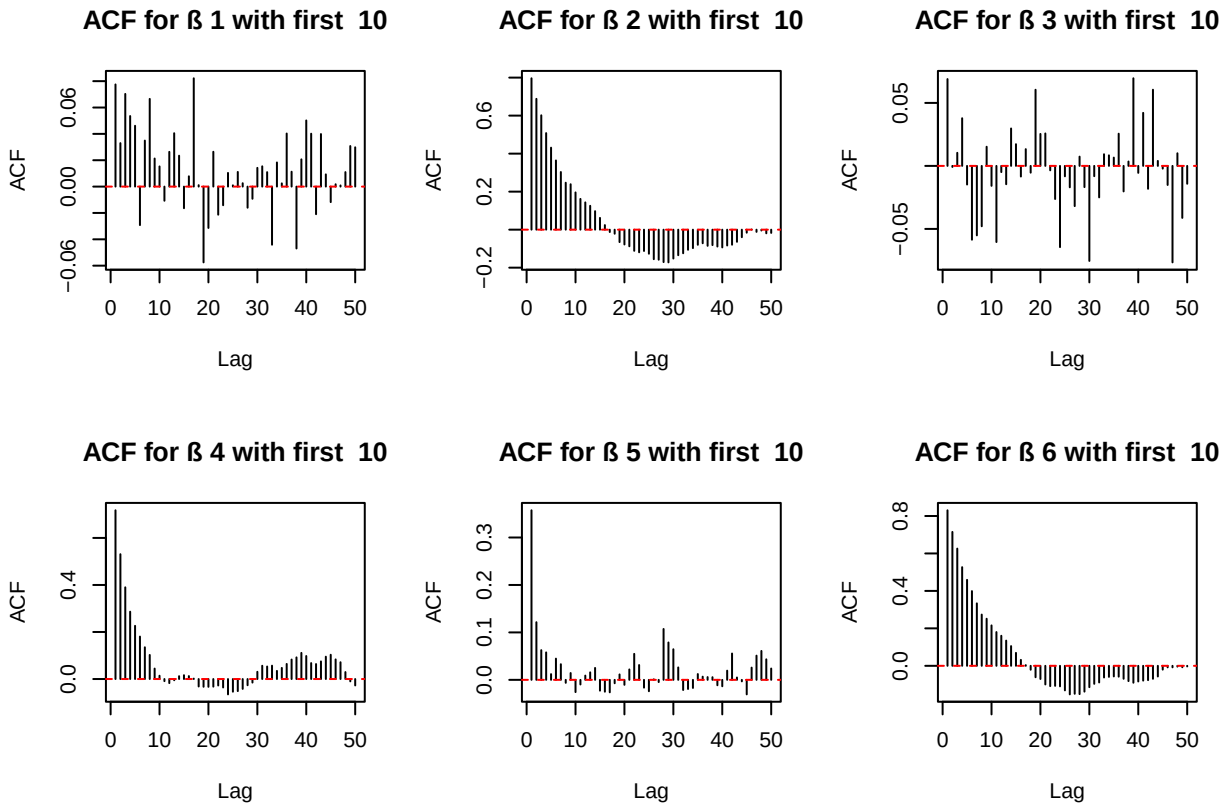
(b) Gibbs Sampler with Varying Data Subset Sizes

Repeat the same task as in (a), but now only use the first m observations of the dataset. Do this for all $m \in \{10, 40, 80\}$.

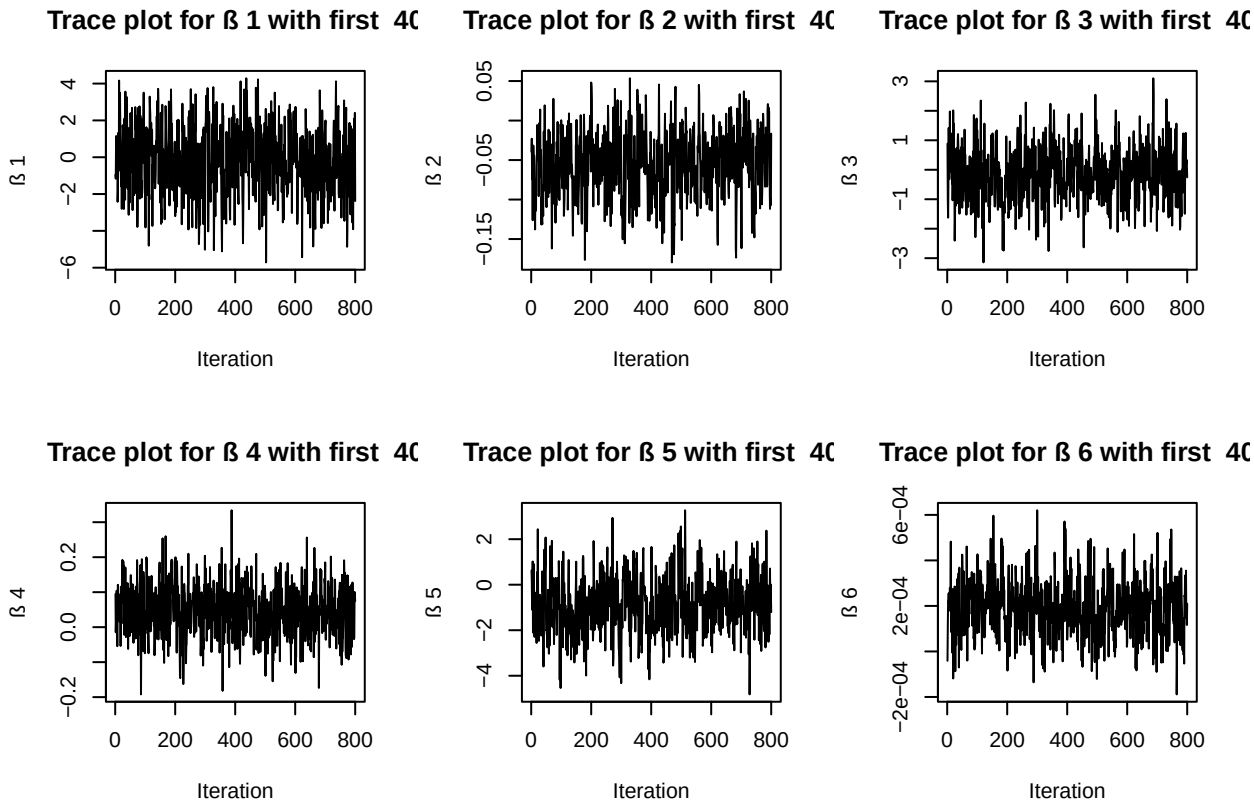
```
m <- c(10, 40, 80)
for (z in 1:3) {
  row_nr <- m[z]
  data = read.csv('Disease.csv')
  x <- as.matrix(data [1:row_nr, 1:6])
  y <- as.matrix(data [1:row_nr, 7])

  # initialize beta
nDraws = 1000
burnin <- 200       # Burn-in period

n <- nrow(x)
p <- ncol(x)

beta <- rep(1, 6) # 5+1 intercept

beta_samples <- matrix(NA, nDraws, p)

set.seed(123)

for (i in 1:nDraws) {
  omega <- rpg(n, 1, x %*% beta)
  k <- y - 1 / 2
  B <- 9 * diag(6)
  b <- rep(0, p)
  # Conditional posterior for beta

  v_omg <- solve(t(x) %*% diag(omega) %*% x + solve(B))
  m_omg <- v_omg %*% (t(x) %*% k + solve(B) %*% b)

  beta <- as.numeric(rmvnorm(1, mean = m_omg, sigma = v_omg))
  beta_samples[i, ] <- beta
}
# Discard burn-in
beta_samples <- beta_samples[(burnin + 1):nDraws, ]
nPost <- nrow(beta_samples)

par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    beta_samples[, i],
    type = 'l',
    main = paste("Trace plot for  ", i , "with first ", row_nr),
    xlab = "Iteration",
    ylab = paste(" ", i)
  )
}
```

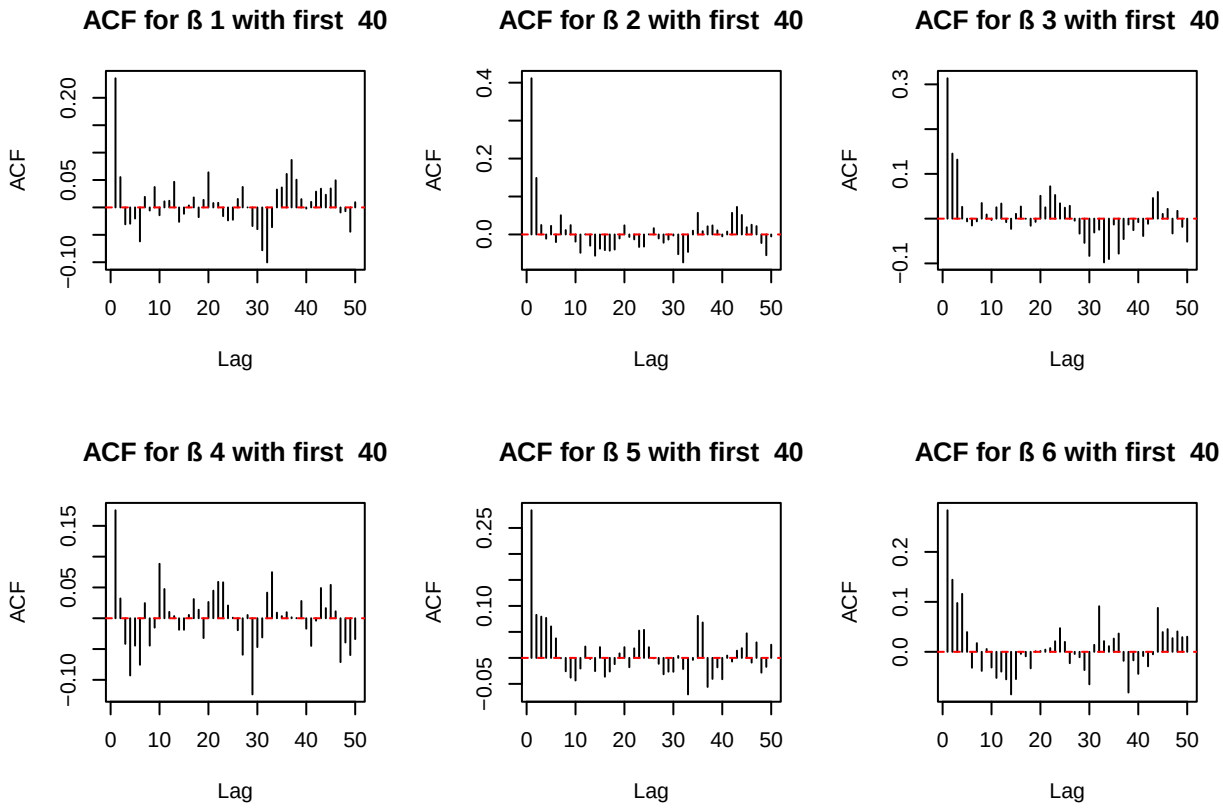
```r
max_lag <- 50  # set the max lag
acfs <- matrix(NA, max_lag, p)

for (i in 1:p) {
  chain <- beta_samples[, i] - mean(beta_samples[, i])
  var_chain <- mean(chain ^ 2)

  for (j in 1:max_lag) {  # j as lag
    # calculate the autocorrelation factor
    # 使用 nPost - j 作为有效样本数更合理
    acfs[j, i] <- sum(chain[1:(nPost - j)] * chain[(j + 1):nPost]) / (nPost-j) / var_chain

  }

}

# or can be done with acf()
acfs <- matrix(NA, nrow = max_lag, ncol = p)

for (i in 1:p) {
  # 自动去均值并计算 acf，type = "correlation" 返回的是标准化后的自相关系数
  acf_res <- acf(beta_samples[, i], lag.max = max_lag, plot = FALSE, type = "correlation")
  acfs[, i] <- acf_res$acf[-1]  # 去掉 lag 0
}


par(mfrow = c(2, 3))  # 6 pics

for (i in 1:p) {
  plot(
    1:max_lag,
    acfs[, i],
    type = 'h',
    main = paste("ACF for  ", i , "with first ", row_nr),
    xlab = "Lag",
    ylab = "ACF"
  )
  abline(h = 0, col = "red", lty = 2)
}

# inefficiency factor (IF)
IFs <- numeric(p)

for (i in 1:p) {
  IFs[i] <- 1 + 2 * sum(acfs[, i])
  cat(paste("IF for  ", i, "=", round(IFs[i], 2), "with first ", row_nr, "\n"))
}

}
```


Trace plot for ß 1 with first 10
ß 1
−5
0
5
0
200
400
600
800
Iteration
Trace plot for ß 2 with first 10
ß 2
−2.0
−1.0
0.0
0
200
400
600
800
Iteration
Trace plot for ß 3 with first 10
ß 3
−5
0
5
10
0
200
400
600
800
Iteration
Trace plot for ß 4 with first 10
ß 4
−1.5
−0.5
0.5
0
200
400
600
800
Iteration
Trace plot for ß 5 with first 10
ß 5
−5
0
5
0
200
400
600
800
Iteration
Trace plot for ß 6 with first 10
ß 6
0.000
0.004
0.008
0
200
400
600
800
Iteration

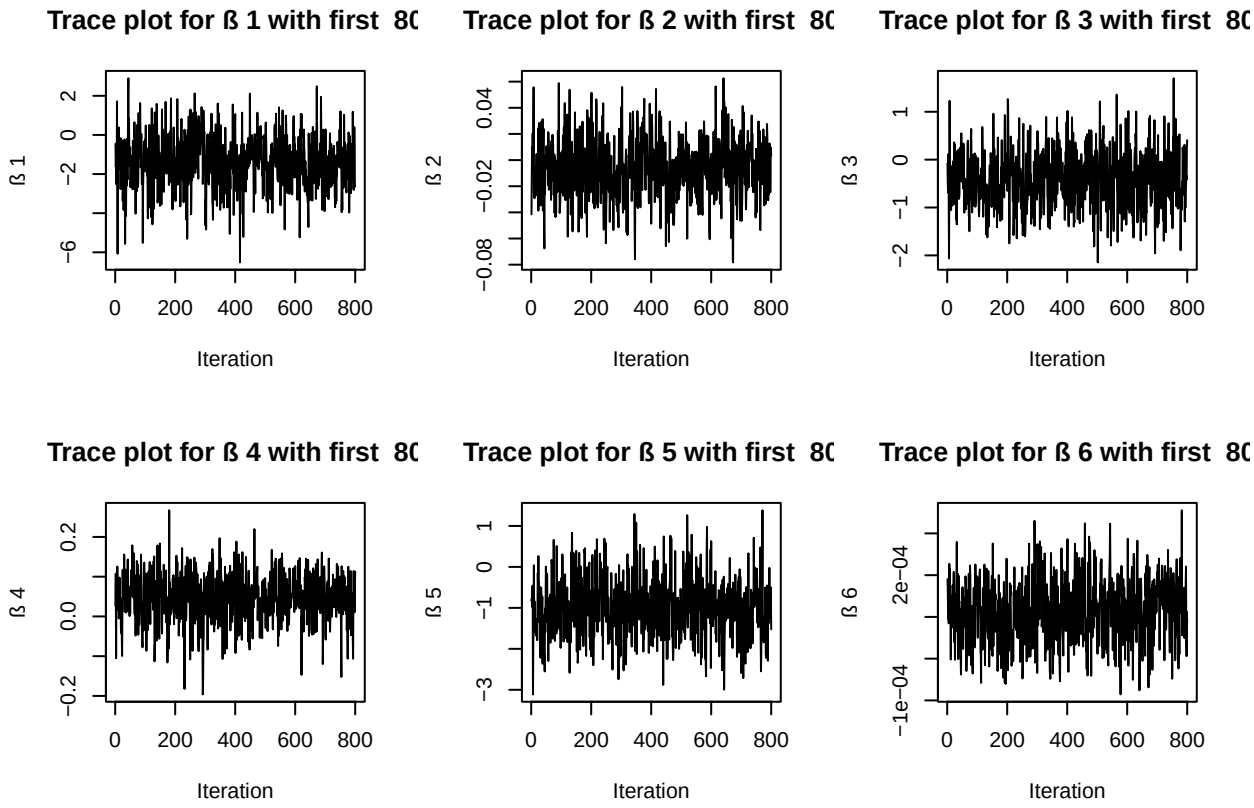


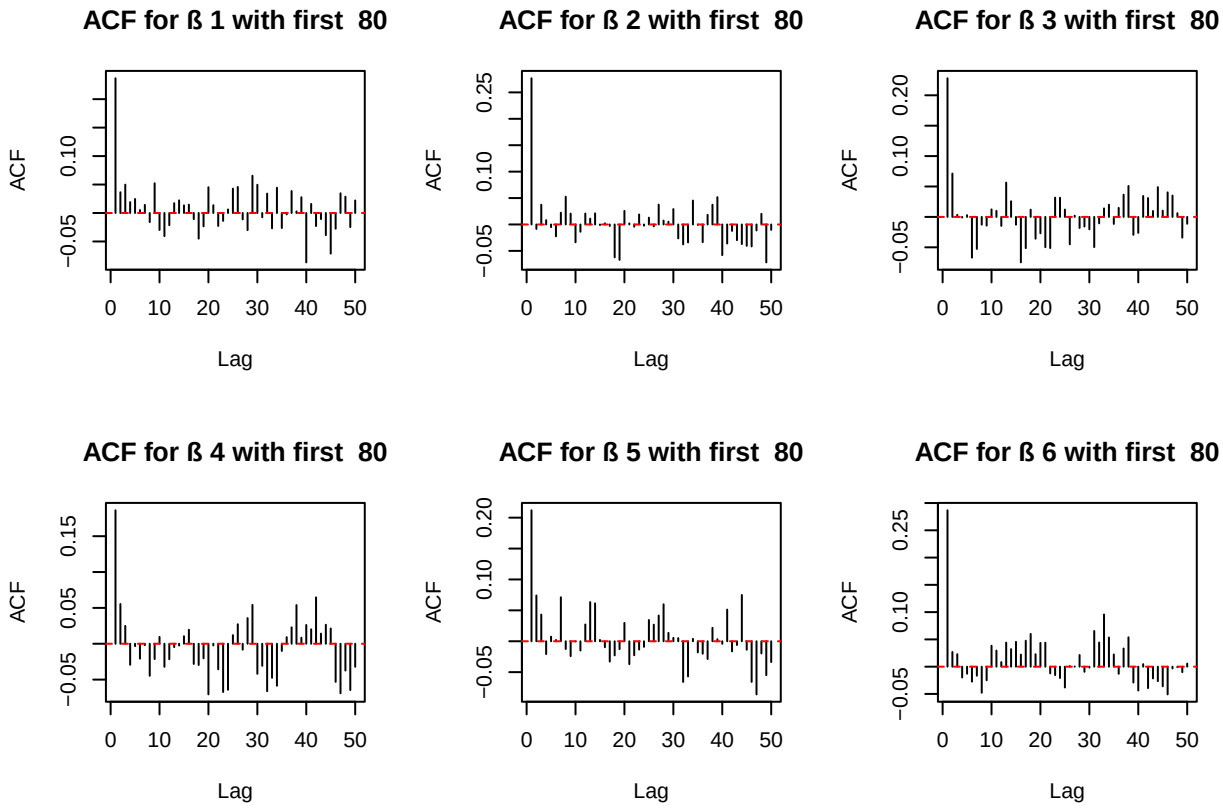

```
## IF for   1 = 2.34 with first  10
## IF for   2 = 5.3 with first  10
## IF for   3 = 0.59 with first  10
## IF for   4 = 7.94 with first  10
## IF for   5 = 3.34 with first  10
## IF for   6 = 6.65 with first  10
```


Trace plot for ß 1 with first 40
ß 1
−6
−2
0
2
4
0
200
400
600
800
Iteration
Trace plot for ß 2 with first 40
ß 2
−0.15
−0.05
0.05
0
200
400
600
800
Iteration
Trace plot for ß 3 with first 40
ß 3
−3
−1
1
3
0
200
400
600
800
Iteration
Trace plot for ß 4 with first 40
ß 4
−0.2
0.0
0.2
0
200
400
600
800
Iteration
Trace plot for ß 5 with first 40
ß 5
−4
−2
0
2
0
200
400
600
800
Iteration
Trace plot for ß 6 with first 40
ß 6
−2e−04
2e−04
6e−04
0
200
400
600
800
Iteration




```
## IF for   1 = 1.85 with first  40
## IF for   2 = 1.75 with first  40
## IF for   3 = 1.73 with first  40
## IF for   4 = 1.02 with first  40
## IF for   5 = 1.95 with first  40
## IF for   6 = 2.06 with first  40
```


Trace plot for ß 1 with first 80
ß 1
−6
−2
0
2
0
200
400
600
800
Iteration
Trace plot for ß 2 with first 80
ß 2
−0.08
−0.02
0.04
0
200
400
600
800
Iteration
Trace plot for ß 3 with first 80
ß 3
−2
−1
0
1
0
200
400
600
800
Iteration
Trace plot for ß 4 with first 80
ß 4
−0.2
0.0
0.2
0
200
400
600
800
Iteration
Trace plot for ß 5 with first 80
ß 5
−3
−1
0
1
0
200
400
600
800
Iteration
Trace plot for ß 6 with first 80
ß 6
−1e−04
2e−04
0
200
400
600
800
Iteration




```
## IF for  1 = 1.83 with first  80
## IF for  2 = 1.15 with first  80
## IF for  3 = 1.23 with first  80
## IF for  4 = 0.36 with first  80
## IF for  5 = 1.42 with first  80
## IF for  6 = 2.31 with first  80
```

As for case 1 where we have 10 observations, there is a decreasing tendency noticed for $\beta 4$ and the overall IF is comparatively high, thus it is not considered to converge.

For case 2 and 3 where there are 40 and 80 observations respectively, all plots have reached a stabilization and all parameters have a good IF factor. Thus convergence is thought to be reached.

---

7. Metropolis Random Walk for Poisson regression

Consider the following Poisson regression model:

$$y_i|\beta \sim \text{Poisson}\left(\exp(x_i^T\beta)\right), \quad i = 1, \ldots, n,$$

where y_i is the count for the ith observation in the sample and x_i is the p-dimensional vector with covariate observations for the ith observation.

The dataset contains observations from 700 eBay auctions of coins. The response variable is `nBids` and records the number of bids in each auction. The remaining variables are features/covariates (x):

- **Const**: Intercept
- **PowerSeller**: Equal to 1 if the seller is selling large volumes on eBay
- **VerifyID**: Equal to 1 if the seller is a verified seller by eBay
- **Sealed**: Equal to 1 if the coin was sold in an unopened envelope
- **MinBlem**: Equal to 1 if the coin has a minor defect
- **MajBlem**: Equal to 1 if the coin has a major defect
- **LargNeg**: Equal to 1 if the seller received a lot of negative feedback from customers
- **LogBook**: Logarithm of the book value of the auctioned coin according to expert sellers (Standardized)
- **MinBidShare**: Ratio of the minimum selling price (starting price) to the book value (Standardized).

(a) Maximum Likelihood Estimation and Covariate Significance in Poisson Regression

Obtain the maximum likelihood estimator of β in the Poisson regression model for the eBay data. [Hint: `glm.R`, don't forget that `glm()` adds its own intercept so don't input the covariate `Const`.] Which covariates are significant?

```r
rm(list=ls())

data <-read.table('eBayNumberOfBidderData_2025.dat', header = FALSE, skip = 1)

colnames(data) <- c("nBids", "Const", "PowerSeller", "VerifyID", "Sealed", "MinBlem", "MajBlem", "LargN

# Fit Poisson regression model (excluding Const)

model <- glm(nBids ~ . - Const, family = poisson, data = data)
summary(model)
```

```
##
## Call:
## glm(formula = nBids ~ . - Const, family = poisson, data = data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.08534    0.03592  30.213  < 2e-16 ***
## PowerSeller -0.02833    0.04433  -0.639  0.52280
## VerifyID    -0.34902    0.13008  -2.683  0.00729 **
## Sealed       0.50101    0.06676   7.504 6.18e-14 ***
## MinBlem     -0.12189    0.08388  -1.453  0.14619
## MajBlem     -0.24884    0.09865  -2.523  0.01165 *
## LargNeg      0.03610    0.07075   0.510  0.60987
## LogBook     -0.07280    0.03505  -2.077  0.03783 *
## MinBidShare -1.76665    0.08292 -21.306  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 1407.17  on 699  degrees of freedom
## Residual deviance:  593.76  on 691  degrees of freedom
## AIC: 2529.5
##
## Number of Fisher Scoring iterations: 5
```

Lower P value means higher significance, as is shown, significant covariants are VerifyID and MinBidShare.

?Derivation of the Log-Posterior in Poisson Regression We model the response variable

$$y_i \in \mathbb{N}$$

using a Poisson distribution:

$$y_i \mid x_i, \beta \sim \text{Poisson}(\lambda_i), \quad \text{where } \lambda_i = \exp(x_i^\top \beta).$$

The probability mass function (pmf) of the Poisson distribution is:

$$p(y_i \mid \lambda_i) = \frac{\lambda_i^{y_i} e^{-\lambda_i}}{y_i!}.$$

Substituting

$$\lambda_i = \exp(x_i^\top \beta)$$

, we obtain:

$$p(y_i \mid x_i, \beta) = \frac{\left(\exp(x_i^\top \beta)\right)^{y_i} \exp\left(-\exp(x_i^\top \beta)\right)}{y_i!}.$$

The **log-likelihood** for a single observation becomes:

$$\log p(y_i \mid x_i, \beta) = y_i (x_i^\top \beta) - \exp(x_i^\top \beta) - \log(y_i!).$$

Summing over all n observations, the total log-likelihood is:

$$\log p(y \mid \beta, X) = \sum_{i=1}^{n} \left[y_i (x_i^\top \beta) - \exp(x_i^\top \beta) - \log(y_i!) \right].$$

Since

$$\log(y_i!)$$

does not depend on

$$\beta$$

, it can be omitted in optimization or posterior sampling. Thus, we use the simplified log-likelihood:

$$\log p(y \mid \beta, X) \propto \sum_{i=1}^{n} \left[y_i (x_i^\top \beta) - \exp(x_i^\top \beta) \right].$$

Prior

We place a Gaussian prior on

$$\beta$$

:

$$\beta \sim \mathcal{N}(0, g(X^\top X)^{-1}),$$

where g = 100 is a scalar. The log prior is:

$$\log p(\beta) = -\frac{1}{2}\beta^\top \left(\frac{1}{g}X^\top X\right)\beta + \text{const.}$$

Log-Posterior

Combining likelihood and prior, we get the **log-posterior** up to a constant:

$$\log p(\beta \mid y, X) \propto \sum_{i=1}^{n} [y_i(x_i^\top \beta) - \exp(x_i^\top \beta)] - \frac{1}{2}\beta^\top \left(\frac{1}{g}X^\top X\right)\beta.$$

This is the function we maximize (e.g., with `optim()`) or use in MCMC sampling.

```
LogPostPoisson <- function(beta, y, X) {
  beta <- as.numeric(beta)
  lin_pred <- X %*% beta
  log_lik <- sum(y * lin_pred - exp(lin_pred))

  p <- length(beta)
  log_prior <- dmvnorm(beta, rep(0, p), 100 * solve(t(X) %*% X), log = TRUE)

  return(log_lik + log_prior)
}
```

This function returns the log-posterior (up to a constant), combining both the likelihood and the g-prior.

(b) Bayesian Analysis: Multivariate Normal Approximation of the Posterior

Let's do a Bayesian analysis of the Poisson regression. Let the prior be $\beta \sim \mathcal{N}(0, 100 \cdot (X^\top X)^{-1})$, where X is the $n \times p$ covariate matrix. This is a commonly used prior, which is called Zellner's g-prior. Assume first that the posterior density is approximately multivariate normal:

$$\beta \mid y \sim \mathcal{N}(\tilde{\beta}, J_y^{-1}(\tilde{\beta})),$$

where $\tilde{\beta}$ is the posterior mode and $J_y(\tilde{\beta})$ is the negative Hessian at the posterior mode.

$\tilde{\beta}$ and $J_y(\tilde{\beta})$ can be obtained by numerical optimization (`optim.R`) exactly like you already did for the logistic regression in Lab 2 (but with the log posterior function replaced by the corresponding one for the Poisson model, which you have to code up).

```
y<- matrix(data$nBids, ncol=1)
x<- as.matrix(data[, 2:10])

LogPostPoisson <- function(beta,y,X){
  beta <- as.numeric(beta)
  lin_pred <- X %*% beta
  p = length(beta)

  # Poisson log-likelihood
  log_lik <- sum(y * lin_pred - exp(lin_pred))
```

```
  # Zellner's g-prior: beta ~ N(0, 100 * (X^T X)^-1)
  log_prior = dmvnorm(beta, rep(0, p), 100*solve(t(X)%*%X), log=TRUE);
  return(log_lik + log_prior)
}

OptimRes <- optim(par = rep(0, ncol(x)),
                  fn = LogPostPoisson,
                  X = x, y = y,
                  method = "BFGS",
                  control = list(fnscale = -1, maxit = 1000),
                  hessian = TRUE)
approxPostMode <- matrix(OptimRes$par,1,9)
cov_matrix <- solve(-OptimRes$hessian)# J^{-1}
```

(c) Metropolis Random Walk Sampling and MCMC Convergence Assessment

Let's simulate from the actual posterior of β using the Metropolis algorithm and compare the results with the approximate results in (b). Program a general function that uses the Metropolis algorithm to generate random draws from an arbitrary posterior density. In order to show that it is a general function for any model, we denote the vector of model parameters by θ. Let the proposal density be the multivariate normal density mentioned in Lecture 8 (random walk Metropolis):

$$\theta^{(p)} \mid \theta^{(i-1)} \sim \mathcal{N}(\theta^{(i-1)}, c \cdot \Sigma),$$

$P(y_i = 1) = \sigma(x_i^\top \beta)$

where $\Sigma = J_y^{-1}(\tilde{\beta})$ was obtained in (b). The value c is a tuning parameter and should be an input to your Metropolis function. The user of your Metropolis function should be able to supply her own posterior density function, not necessarily for the Poisson regression, and still be able to use your Metropolis function. This is not so straightforward, unless you have come across function objects in R. The note `HowToCodeRWM.pdf` in Lisam describes how you can do this in R.

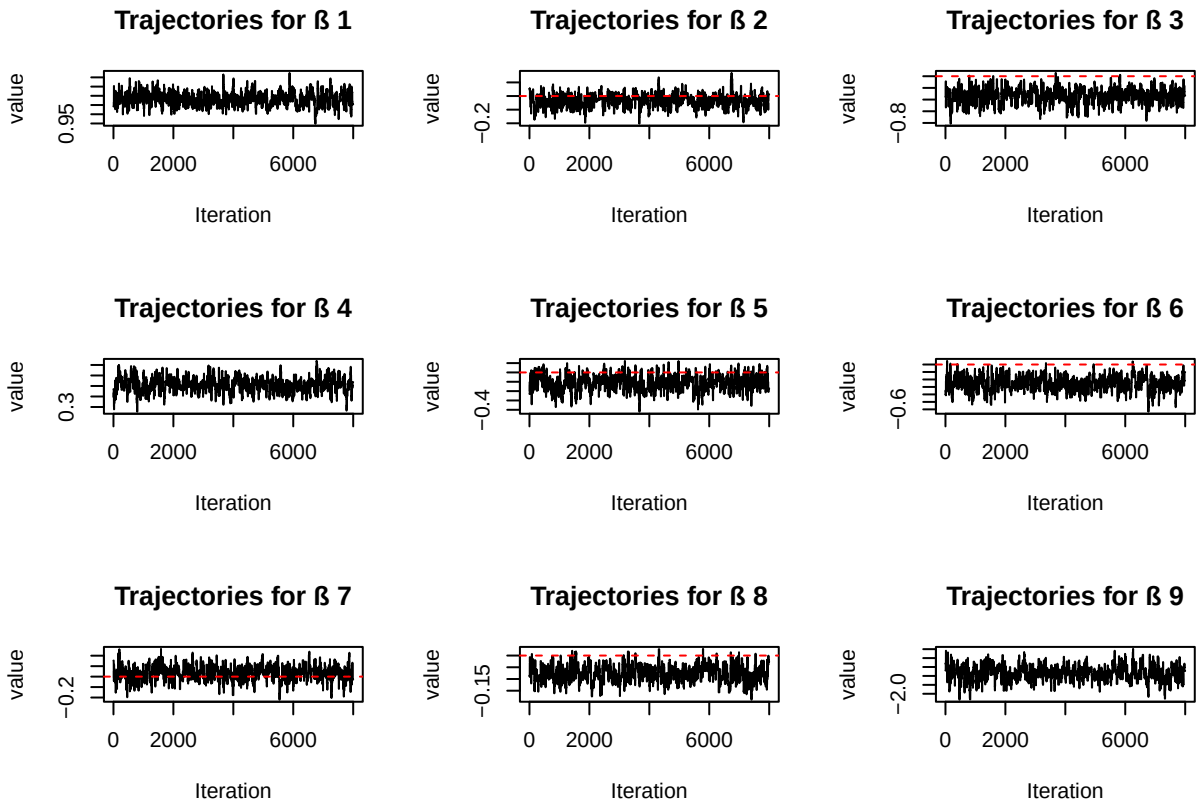
Now, use your new Metropolis function to sample from the posterior of β in the Poisson regression for the eBay dataset. Assess MCMC convergence by graphical methods.

```
mp_sim <- function(c, LogPostFunc, ndraws, theta_init, sigma) {
  p <- length(theta_init)
  theta_mat <- matrix(NA, ndraws, p)
  theta_mat[1, ] <- theta_init

  for (i in 2:ndraws) {
    theta_new <- rmvnorm(1, theta_mat[i - 1, ], c * sigma)

    log_post_new <- LogPostFunc(theta_new, y = y, X = x)
    log_post_old <- LogPostFunc(theta_mat[i - 1, ], y = y, X = x)

    acc_prob <- min(1, exp(log_post_new - log_post_old))

    if (acc_prob > runif(1)) {
      theta_mat[i, ] <- theta_new
    } else {
```

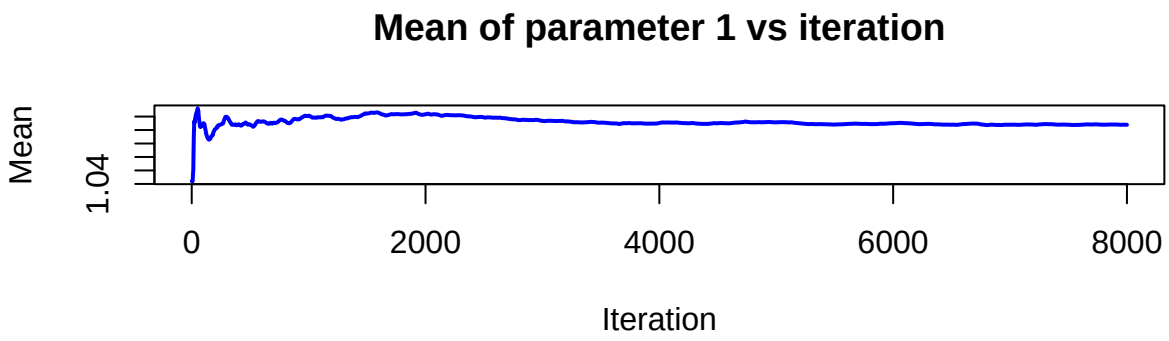
```
      theta_mat[i, ] <- theta_mat[i - 1, ]
    }
  }

  return(theta_mat)
}
```

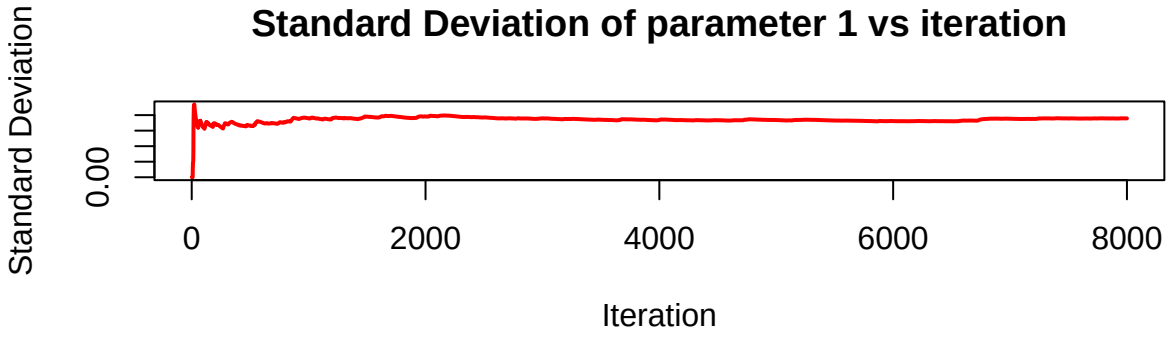
```
set.seed(123)
possion_samp <- mp_sim(1,LogPostFunc = LogPostPoisson,
                       ndraws=10000,
                       theta_init=OptimRes$par,
                       sigma=cov_matrix)
```

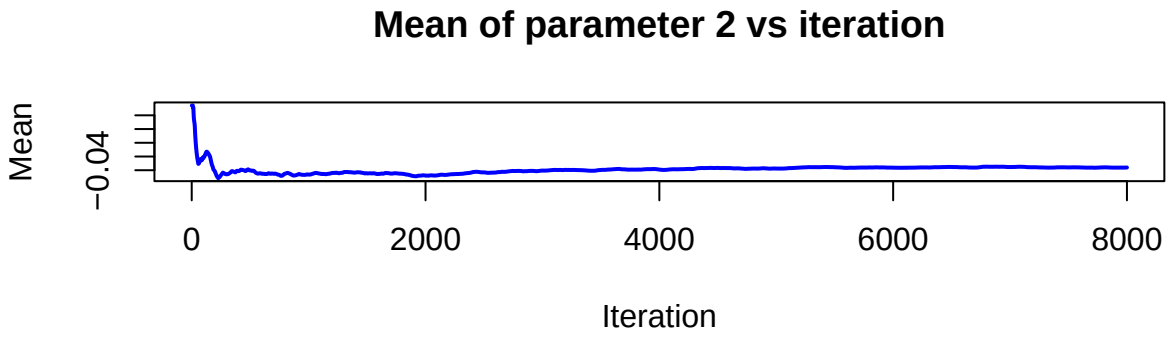
```
par(mfrow = c(3, 3))  # 9 pics
p <- length(OptimRes$par)
burnin <- 2000

for (i in 1:p) {
  plot(
    1:(dim(possion_samp)[1]-burnin),
    possion_samp[(burnin+1):nrow(possion_samp), i],
    type = 'l',
    main = paste("Trajectories for  ", i),
    xlab = "Iteration",
    ylab = "value"
  )
  abline(h = 0, col = "red", lty = 2)
}
```

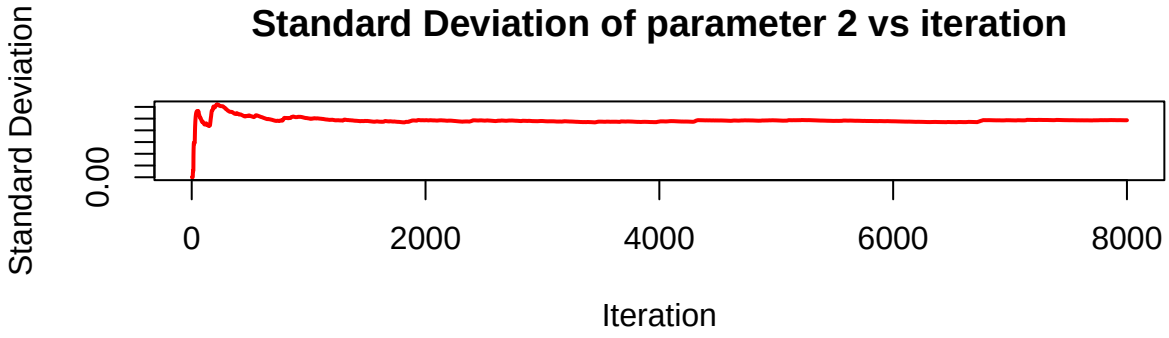


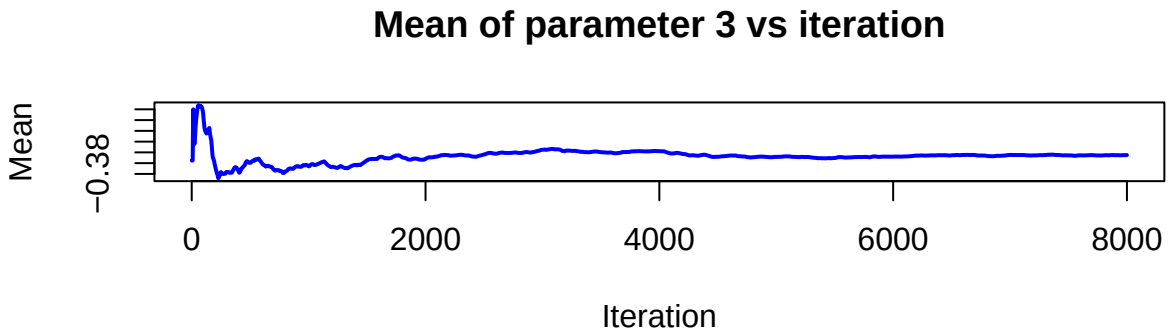

```
ndraws <- nrow(possion_samp)
draw_plot <- ndraws-burnin
for (j in 1:9) {
  param_index <- j  # choose the ith param

mean_vec <- numeric(draw_plot)
sd_vec <- numeric(draw_plot)

for (i in 1:draw_plot) {
  samples_so_far <- possion_samp[(burnin+1):(i+burnin),param_index]
  mean_vec[i] <- mean(samples_so_far)
  sd_vec[i] <- sd(samples_so_far)
}


par(mfrow=c(2,1))

plot(1:draw_plot, mean_vec, type='l', col='blue', lwd=2,
     xlab='Iteration', ylab='Mean',
     main=paste('Mean of parameter', param_index, 'vs iteration'))

plot(1:draw_plot, sd_vec, type='l', col='red', lwd=2,
     xlab='Iteration', ylab='Standard Deviation',
     main=paste('Standard Deviation of parameter', param_index, 'vs iteration'))
}
```

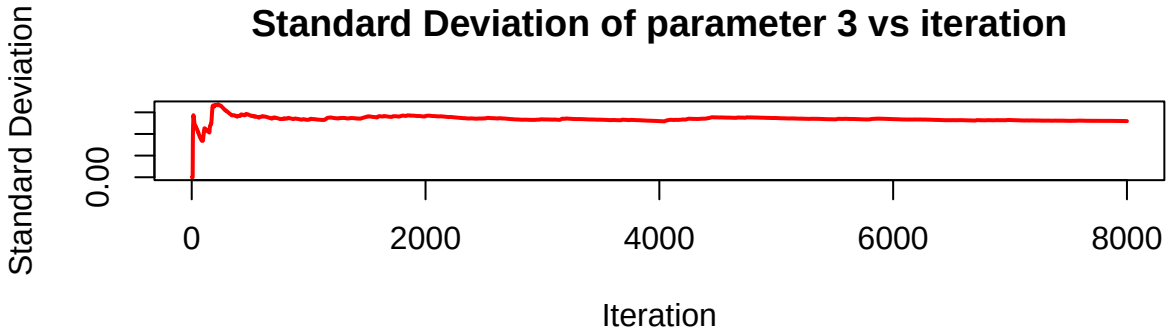

Mean of parameter 1 vs iteration
Mean
1.04
0
2000
4000
6000
8000
Iteration

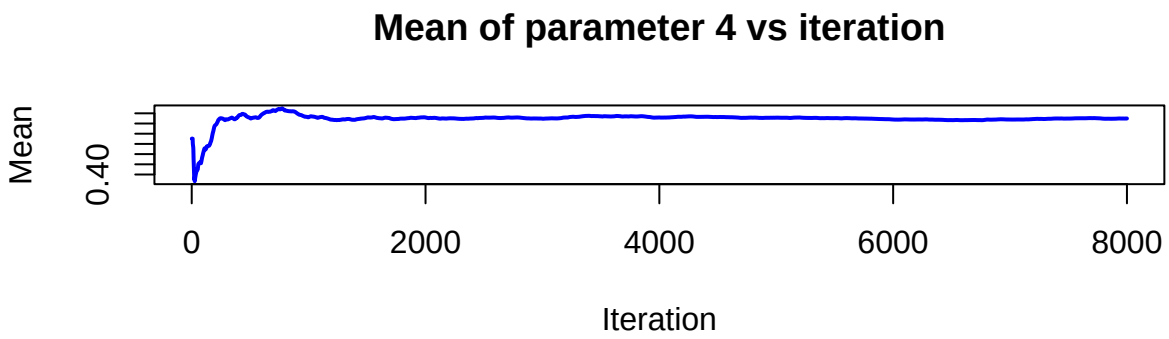

Standard Deviation of parameter 1 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration

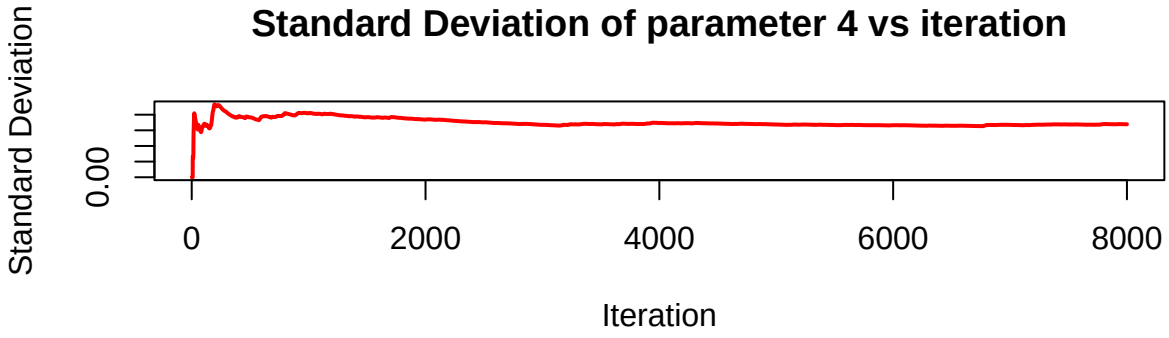

Mean of parameter 2 vs iteration
Mean
−0.04
0
2000
4000
6000
8000
Iteration

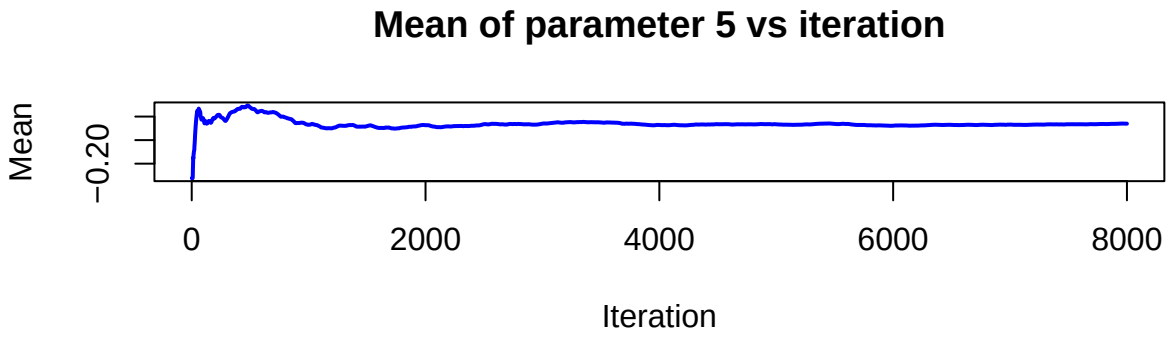

Standard Deviation of parameter 2 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration

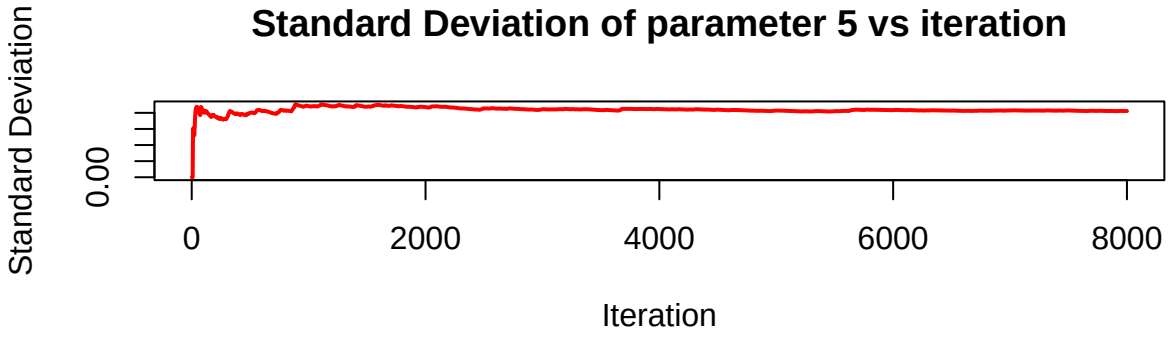

Mean of parameter 3 vs iteration
Mean
−0.38
0
2000
4000
6000
8000
Iteration

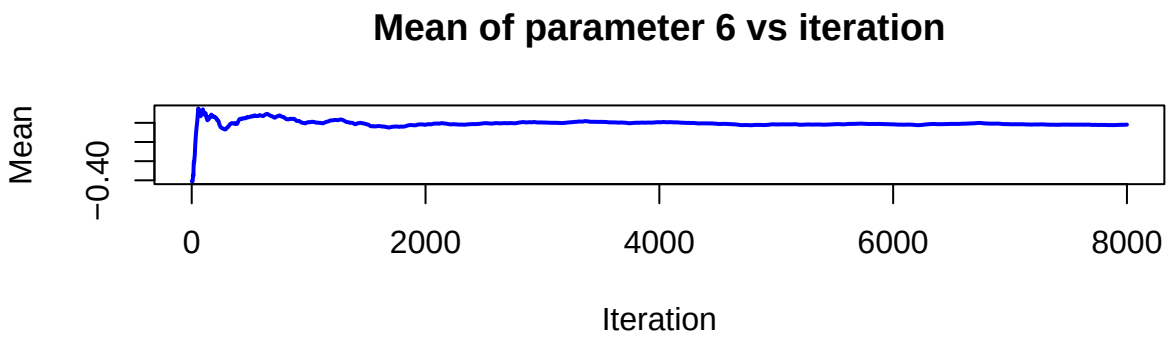

Standard Deviation of parameter 3 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration

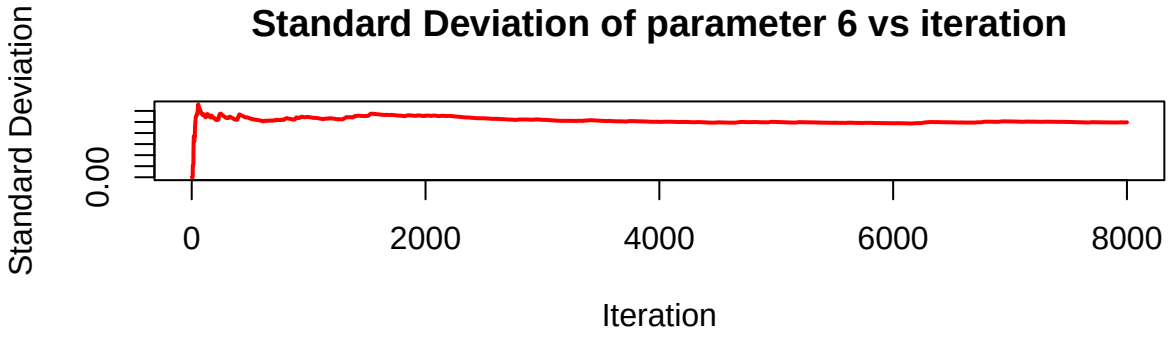

Mean of parameter 4 vs iteration
Mean
0.40
0
2000
4000
6000
8000
Iteration

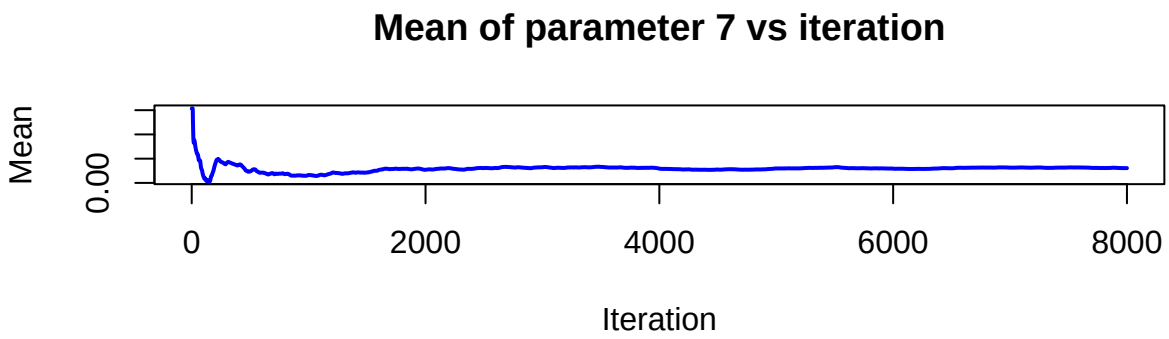

Standard Deviation of parameter 4 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration

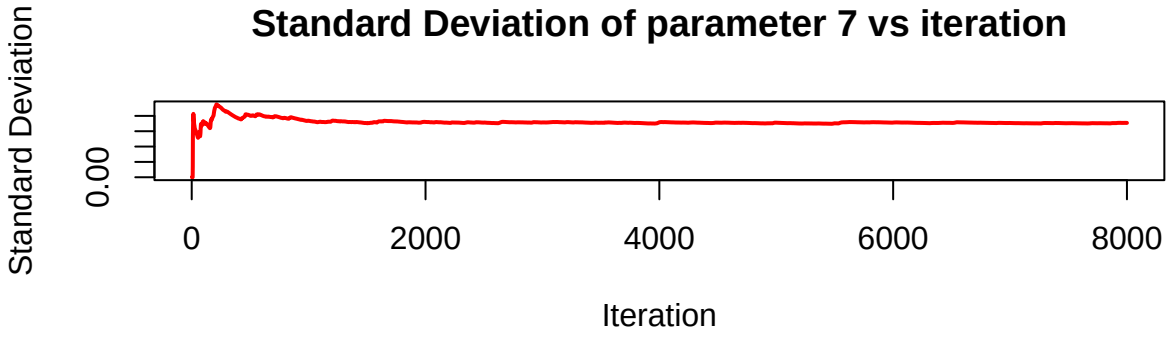

Mean of parameter 5 vs iteration
Mean
−0.20
0
2000
4000
6000
8000
Iteration

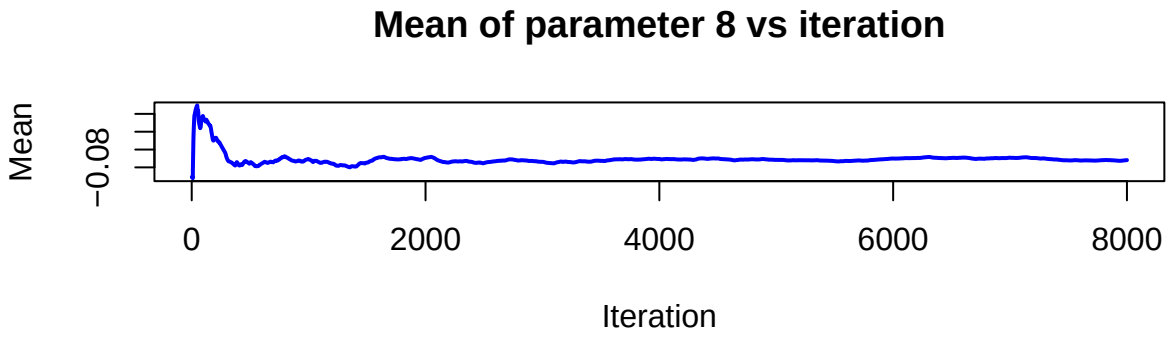

Standard Deviation of parameter 5 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration

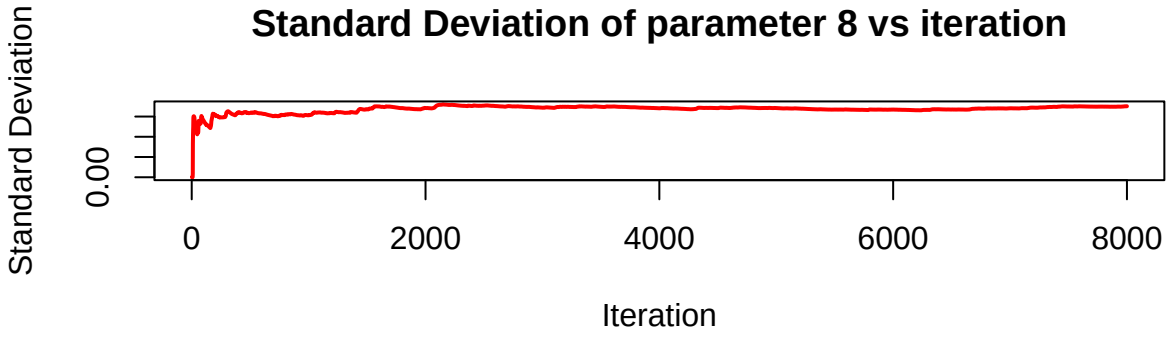

Mean of parameter 6 vs iteration
Mean
−0.40
0
2000
4000
6000
8000
Iteration

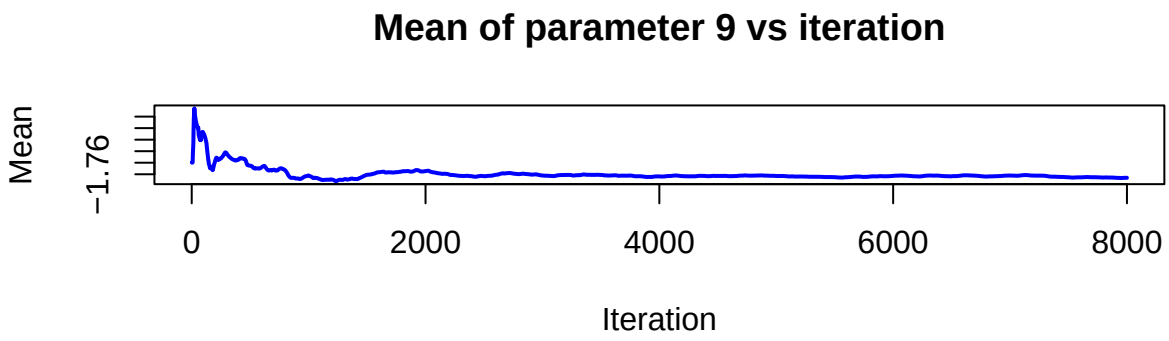

Standard Deviation of parameter 6 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration

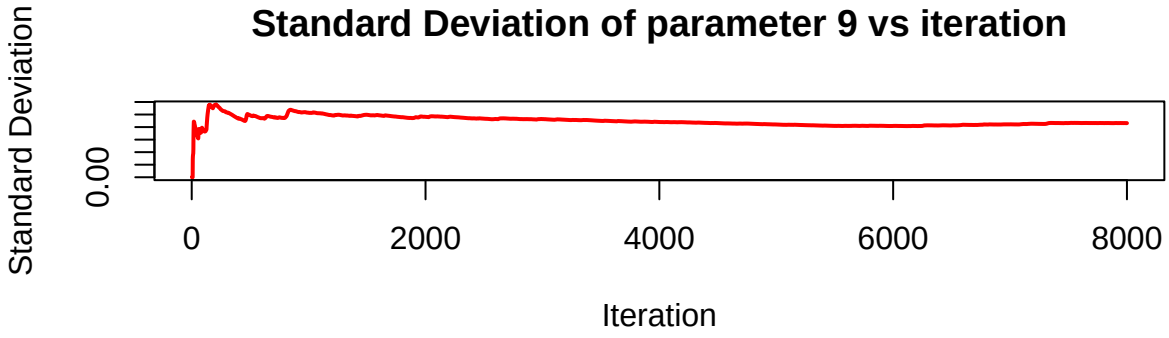

Mean of parameter 7 vs iteration
Mean
0.00
0
2000
4000
6000
8000
Iteration


Standard Deviation of parameter 7 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration


Mean of parameter 8 vs iteration
Mean
−0.08
0
2000
4000
6000
8000
Iteration


Standard Deviation of parameter 8 vs iteration
Standard Deviation
0.00
0
2000
4000
6000
8000
Iteration





Since trajectories and the mean and standard deviation for different parameters are stable, convergence is thus considered reached.

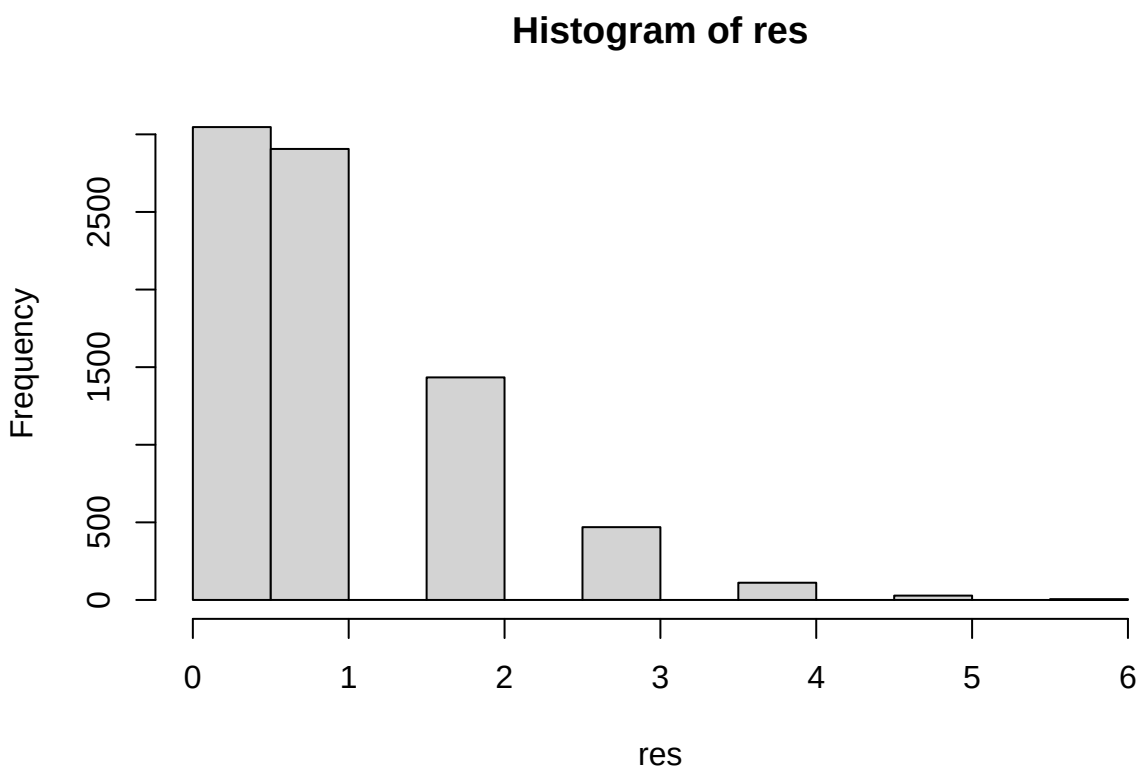
(d) Posterior Predictive Distribution for a New Auction

Use the MCMC draws from (c) to simulate from the predictive distribution of the number of bidders in a new auction with the characteristics below. Plot the predictive distribution. What is the probability of no bidders in this new auction?

- PowerSeller = 1
- VerifyID = 0
- Sealed = 1
- MinBlem = 0
- MajBlem = 1
- LargNeg = 0
- LogBook = 1.3

- MinBidShare = 0.7

```
bidden <- c(1,1,0,1,0,1,0,1.3,0.7) # intercept as 1 to include it
lambda <- exp(possion_samp[(burnin+1):ndraws,] %*% bidden)

res <-rep(NA,length(lambda))
set.seed(124)
for (i in 1:length(lambda)) {
  res[i] <- rpois(1, lambda[i])
}

hist(res)
```




```
prob_0 <- sum(res ==0)/length(lambda)
print(prob_0)
```

```
## [1] 0.380875
```

8.Time series models in Stan

(a) Simulation and Exploration of the AR(1) Process

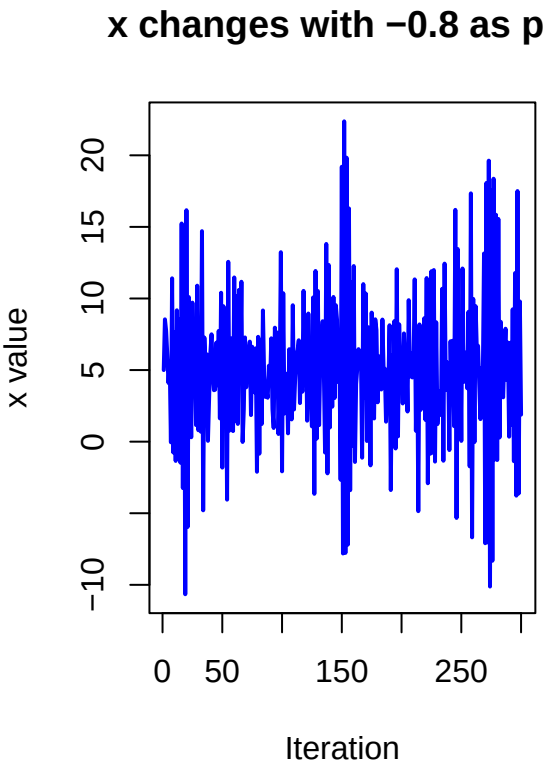
Write a function in R that simulates data from the AR(1)-process

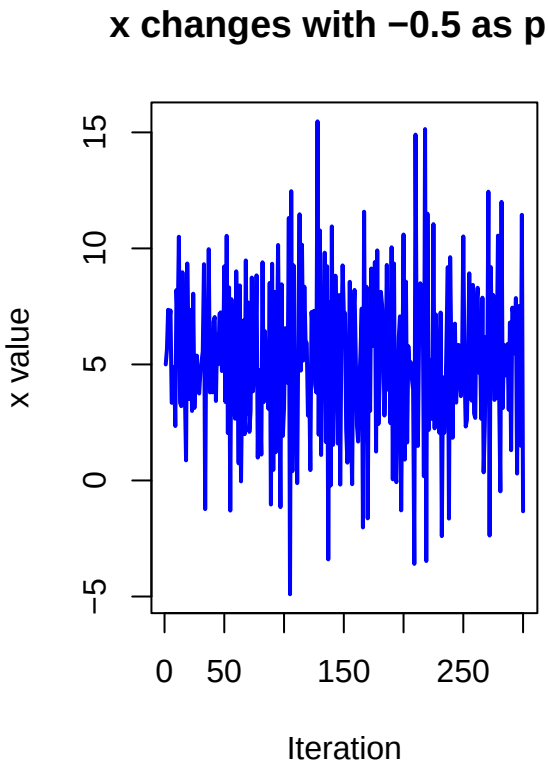
$$x_t = \mu + \phi(x_{t-1} - \mu) + \varepsilon_t, \quad \varepsilon_t \overset{iid}{\sim} \mathcal{N}(0, \sigma^2),$$

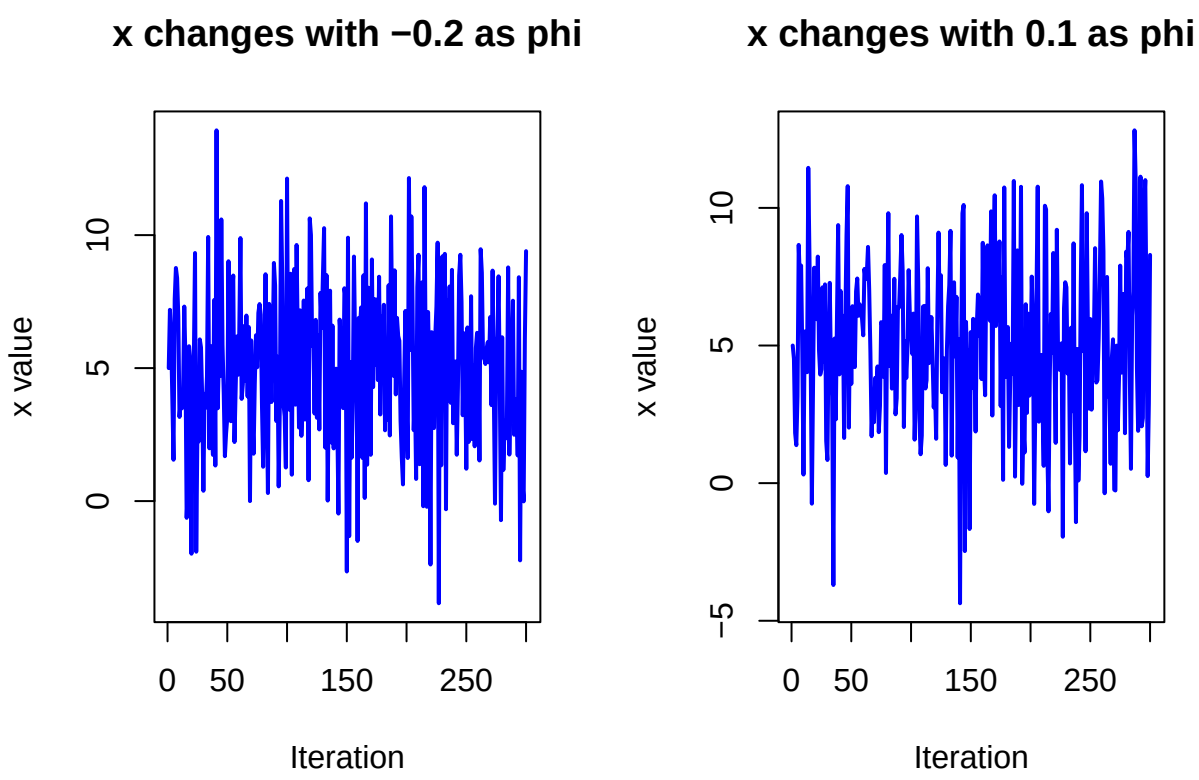
for given values of μ, ϕ and σ^2. Start the process at $x_1 = \mu$ and then simulate values for x_t for $t = 2, 3, \ldots, T$ and return the vector $x_{1:T}$ containing all time points. Use $\mu = 5$, $\sigma^2 = 9$, and $T = 300$ and look at some different realizations (simulations) of $x_{1:T}$ for values of ϕ between -1 and 1 (this is the interval of ϕ where the AR(1)-process is stationary). Include a plot of at least one realization in the report. What effect does the value of ϕ have on $x_{1:T}$?

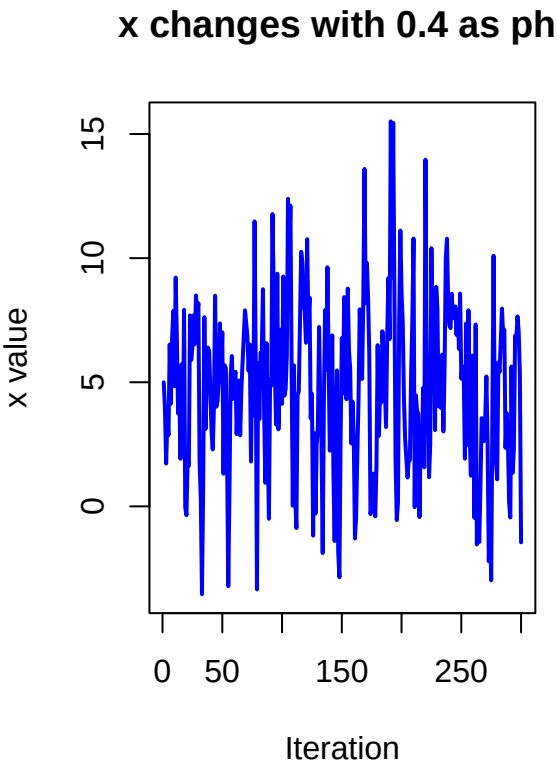
```
rm(list=ls())
#define the function

ar <- function (mu, phi, sig2, T) {
  res<- c()
  x<- mu
  res<-x
  for (t in 2:T) {
    x_new <- mu + phi*(x-mu)+rnorm(1,0,sqrt(sig2))
    res<- c(res,x_new)
    x<- x_new
  }
  return(res)
}

#simulate
set.seed(123)
p <- seq(-1,1,0.1)
res <- matrix(NA,nrow=300, ncol=length(p))
for (i in 1:length(p)) {
  res[,i] <-ar (mu=5, phi=p[i], sig2=9, T=300)

}
```

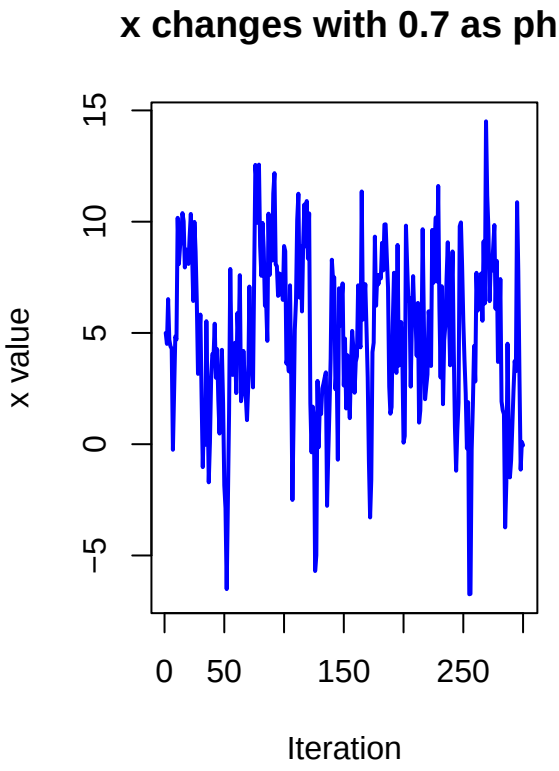
```
nplot <- seq(3,21,3)
par(mfrow = c(1, 2))  # 6 pics
for (i in nplot) {
  plot(res[,i], type='l', col='blue', lwd=2,
     xlab='Iteration', ylab='x value',
     main=paste('x changes with',p[i], 'as phi'))
}
```

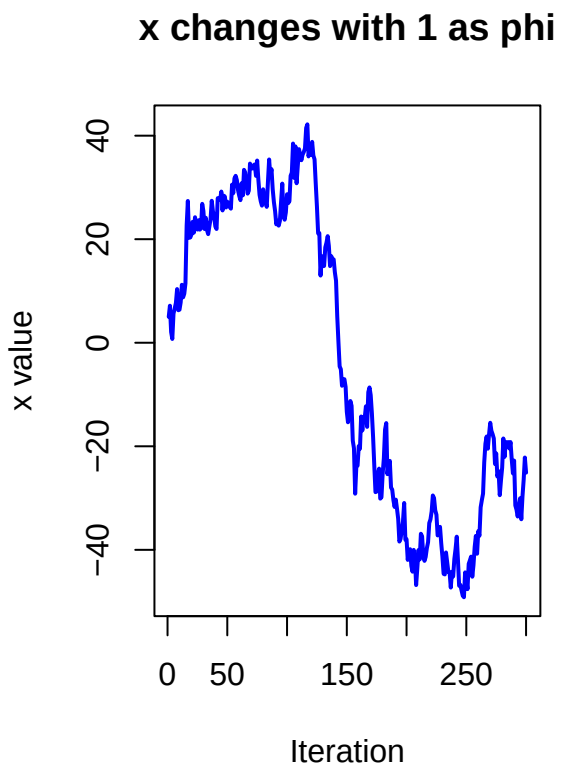

x changes with −0.8 as phi
x value
−10
0
5
10
15
20
0
50
150
250
Iteration


x changes with −0.5 as phi
x value
−5
0
5
10
15
0
50
150
250
Iteration


x changes with −0.2 as phi
x value
0
5
10
0
50
150
250
Iteration
x changes with 0.1 as phi
x value
−5
0
5
10
0
50
150
250
Iteration


x changes with 0.4 as phi
x value
0
5
10
15
0
50
150
250
Iteration


x changes with 0.7 as phi
x value
−5
0
5
10
15
0
50
150
250
Iteration



ϕ controls the strength of autocorrelation. ϕ with small absolute value gives weak correlation and values fulctuate quickly.

(b) Bayesian Inference for AR(1) Parameters using Stan

Use your function from (a) to simulate two AR(1)-processes, $x_{1:T}$ with $\phi = 0.4$ and $y_{1:T}$ with $\phi = 0.98$. Now, treat your simulated vectors as synthetic data, and treat the values of μ, ϕ and σ^2 as unknown parameters. Implement Stan-code that samples from the posterior of the three parameters, using suitable non-informative priors of your choice.

> *Hint:* Look at the time-series models examples in the Stan user's guide/reference manual, and note the different parameterization used here.

i. Report the posterior mean, 95% credible intervals, and the number of effective posterior samples for the three inferred parameters for each of the simulated AR(1)-processes. Are you able to estimate the true values?

ii. For each of the two data sets, evaluate the convergence of the samplers and plot the joint posterior of μ and ϕ. Comments?

```
set.seed(123)
ar1 <- ar(mu=5, phi=0.4, sig2=9, T=300)
ar2 <- ar(mu=5, phi=0.98, sig2=9, T=300)
```

```r
library(rstan)
```

```
## 载入需要的程序包：StanHeaders
```

```
##
## rstan version 2.32.7 (Stan version 2.32.2)
```

```
## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using `reduce_sum()` or `map_rect()` Stan functions,
## change `threads_per_chain` option:
## rstan_options(threads_per_chain = 1)
```

```
## Do not specify '-march=native' in 'LOCAL_CPPFLAGS' or a Makevars file
```

```r
y=ar1
N=length(y)

StanModel = '
data {
  int<lower=0> N; // Number of observations
  vector[N] y; // Number of flowers
}
parameters {
  real mu;
  real<lower=-1, upper=1> phi;
  real<lower=0> sigma;
}
model {
  mu ~ normal(0, 5);
  phi ~ uniform(-1,1);
  sigma ~ normal(3, 3); // Normal with mean 3, st.dev. 3  It is normal (mean, sd_dev) in stan

  for(i in 2:N){
    y[i]~ normal(mu+ phi*(y[i-1] - mu), sigma);
  }
}'



data <- list(N=N, y=y)
warmup <- 50
niter <- 2000
fit <- stan(model_code=StanModel,data=data, warmup=warmup,iter=niter,chains=4)
```

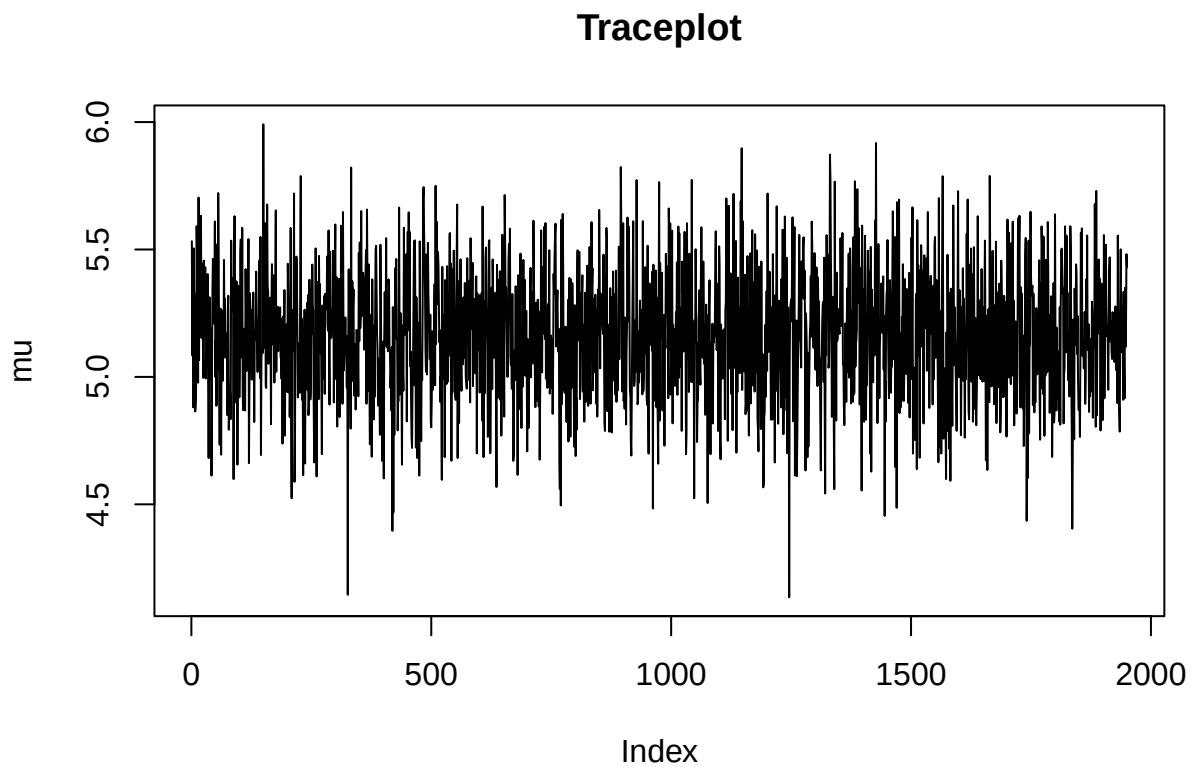
```
## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.
```

```
## Trying to compile a simple C file

## Running "C:/PROGRA~1/R/R-44~1.3/bin/x64/Rcmd.exe" SHLIB foo.c
## using C compiler: 'gcc.exe (GCC) 13.3.0'
## gcc  -I"C:/PROGRA~1/R/R-44~1.3/include" -DNDEBUG   -I"C:/Users/Administrator/AppData/Local/R/win-libi
## cc1.exe: warning: command-line option '-std=c++14' is valid for C++/ObjC++ but not for C
## In file included from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/
##                  from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/
##                  from C:/Users/Administrator/AppData/Local/R/win-library/4.4/StanHeaders/include/stan
##                  from <command-line>:
## C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/src/Core/util/Macros.h
##   679 | #include <cmath>
##       |          ^~~~~~
## compilation terminated.
## make: *** [C:/PROGRA~1/R/R-44~1.3/etc/x64/Makeconf:289: foo.o] Error 1

## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 6e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.6 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: WARNING: There aren't enough warmup iterations to fit the
## Chain 1:          three stages of adaptation as currently configured.
## Chain 1:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 1:          the given number of warmup iterations:
## Chain 1:            init_buffer = 7
## Chain 1:            adapt_window = 38
## Chain 1:            term_buffer = 5
## Chain 1:
## Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 1: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 1: Iteration:  450 / 2000 [ 22%]  (Sampling)
## Chain 1: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 1: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 1: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 1: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 1: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 1: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 1: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1:
## Chain 1:  Elapsed Time: 0.027 seconds (Warm-up)
## Chain 1:                0.554 seconds (Sampling)
## Chain 1:                0.581 seconds (Total)
```

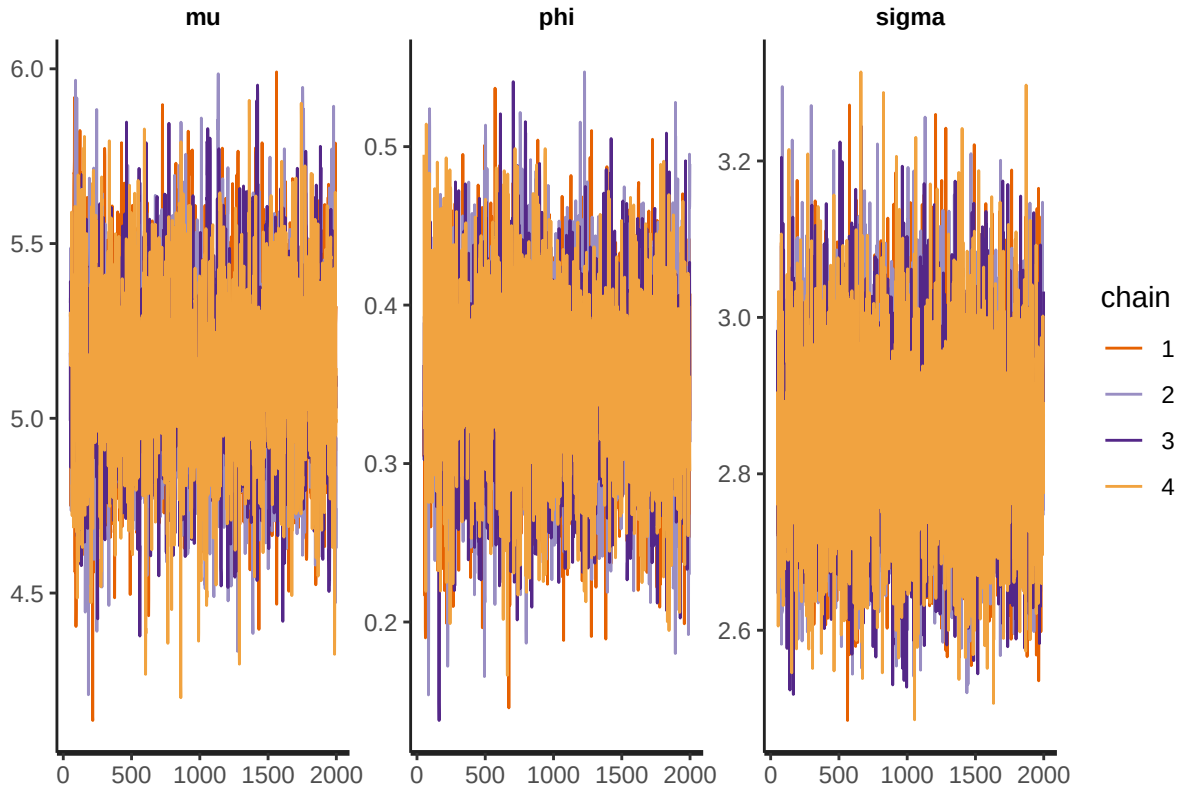
```
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 2.7e-05 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.27 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: WARNING: There aren't enough warmup iterations to fit the
## Chain 2:          three stages of adaptation as currently configured.
## Chain 2:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 2:          the given number of warmup iterations:
## Chain 2:            init_buffer = 7
## Chain 2:            adapt_window = 38
## Chain 2:            term_buffer = 5
## Chain 2:
## Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 2: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 2: Iteration:  450 / 2000 [ 22%]  (Sampling)
## Chain 2: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 2: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 2: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 2: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 2: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 2: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 2: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2:
## Chain 2:  Elapsed Time: 0.029 seconds (Warm-up)
## Chain 2:                0.636 seconds (Sampling)
## Chain 2:                0.665 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 2.9e-05 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.29 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: WARNING: There aren't enough warmup iterations to fit the
## Chain 3:          three stages of adaptation as currently configured.
## Chain 3:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 3:          the given number of warmup iterations:
## Chain 3:            init_buffer = 7
## Chain 3:            adapt_window = 38
## Chain 3:            term_buffer = 5
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 3: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 3: Iteration:  450 / 2000 [ 22%]  (Sampling)
```

```
## Chain 3: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 3: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 3: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 3: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 3: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 3: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 3: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3:
## Chain 3:  Elapsed Time: 0.017 seconds (Warm-up)
## Chain 3:                0.651 seconds (Sampling)
## Chain 3:                0.668 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 2.6e-05 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.26 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: WARNING: There aren't enough warmup iterations to fit the
## Chain 4:          three stages of adaptation as currently configured.
## Chain 4:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 4:          the given number of warmup iterations:
## Chain 4:            init_buffer = 7
## Chain 4:            adapt_window = 38
## Chain 4:            term_buffer = 5
## Chain 4:
## Chain 4: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 4: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 4: Iteration:  450 / 2000 [ 22%]  (Sampling)
## Chain 4: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 4: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 4: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 4: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 4: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 4: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 4: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4:
## Chain 4:  Elapsed Time: 0.03 seconds (Warm-up)
## Chain 4:                0.661 seconds (Sampling)
## Chain 4:                0.691 seconds (Total)
## Chain 4:
```

```r
# Print the fitted model
#print(fit,digits_summary=3)
# Extract posterior samples
postDraws <- extract(fit)
# Do traceplots of the first chain
par(mfrow = c(1,1))
plot(postDraws$mu[1:(niter-warmup)],type="l",ylab="mu",main="Traceplot")
```

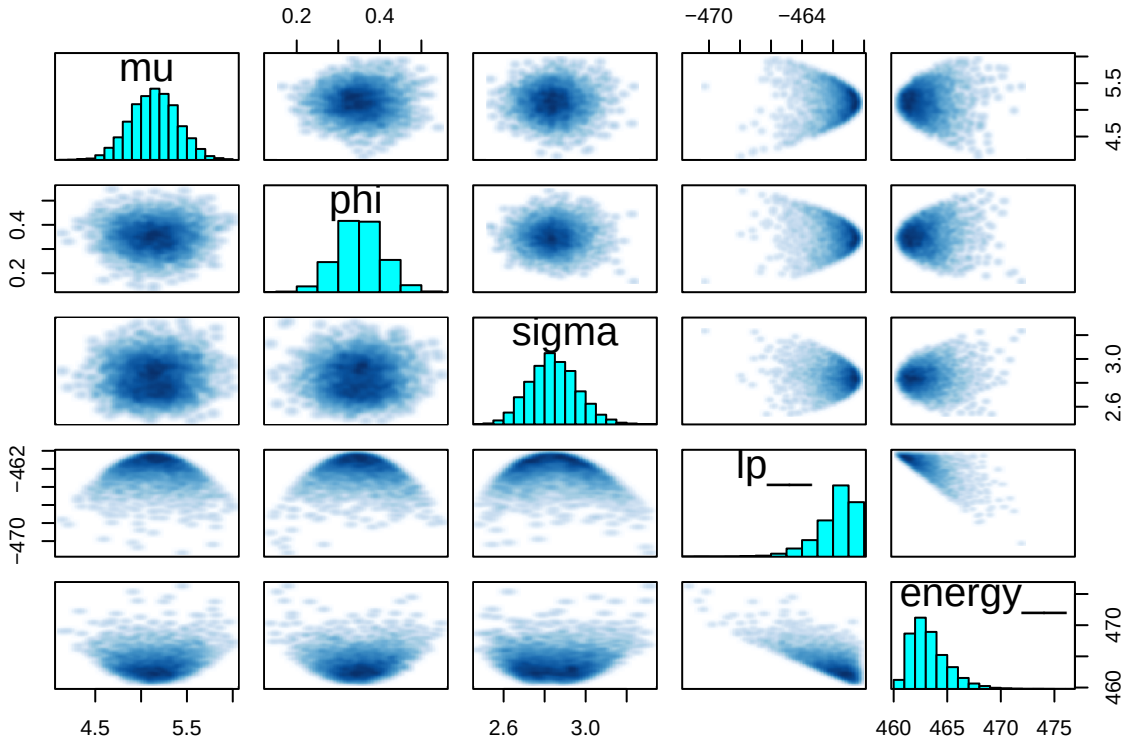



```
# Do automatic traceplots of all chains
traceplot(fit)
```




```
# Bivariate posterior plots
pairs(fit)
```

```
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
```

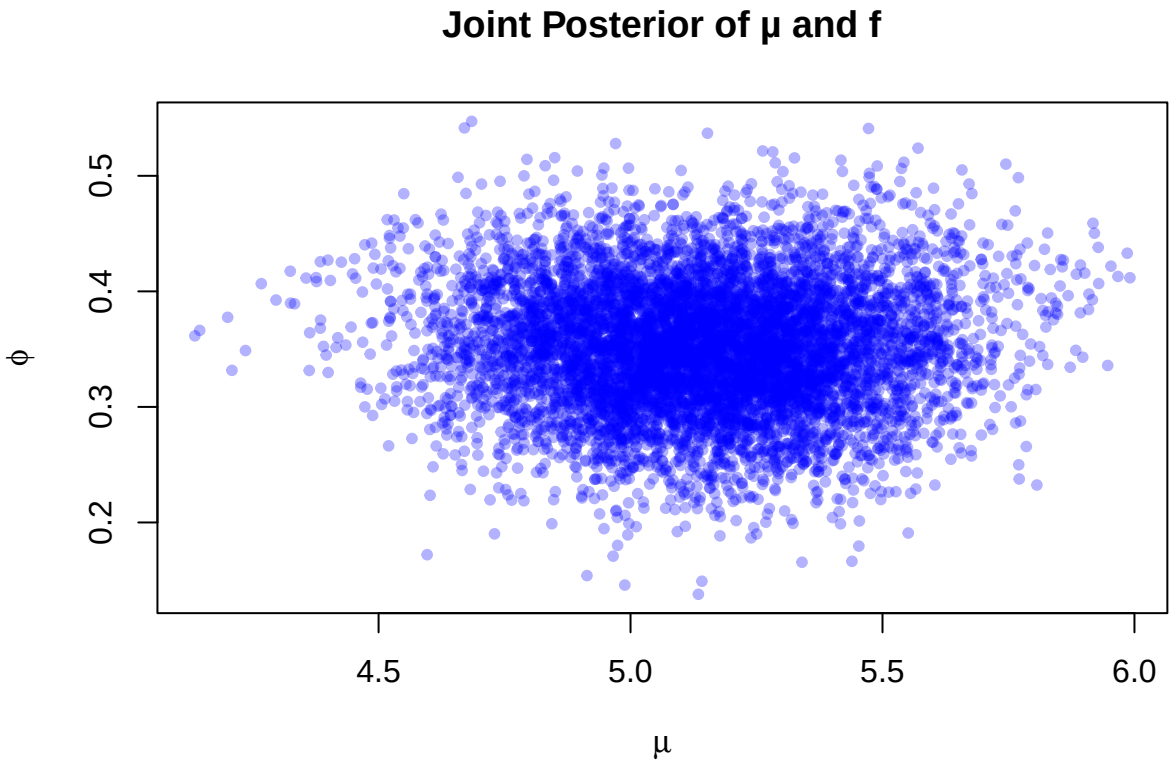



```
summary_fit <- summary(fit, pars = c("mu", "phi", "sigma"))$summary
# extract the mean, 95% CI and effective posterior sample
posterior_summary <- summary_fit[, c("mean", "2.5%", "97.5%", "n_eff" , "Rhat")]
print(round(posterior_summary, 3))
```

```
##         mean  2.5% 97.5%    n_eff  Rhat
## mu    5.144 4.639 5.640 2006.820 1.002
## phi   0.351 0.246 0.459 4176.580 1.001
## sigma 2.844 2.628 3.088 8619.746 1.000
```

Mean, 95% CI and effective posterior sample number are shown as above. The simulated parameter values are quite close to the true parameter values and the true parameter lies in the 95% CI of simulated parameter values, thus we are able to estimate the true values.

```
posterior_samples <- as.data.frame(rstan::extract(fit, pars = c("mu", "phi")))

plot(posterior_samples$mu, posterior_samples$phi,
     xlab = expression(mu), ylab = expression(phi),
     main = "Joint Posterior of   and  ", pch = 20, col = rgb(0, 0, 1, 0.3))
```



```
y=ar2
N=length(y)

StanModel = '
data {
  int<lower=0> N; // Number of observations
  vector[N] y; // Number of flowers
}
parameters {
  real mu;
  real<lower=-1, upper=1> phi;
  real<lower=0> sigma;
}
model {
  mu ~ normal(0, 5);
  phi ~ uniform(-1,1);
  sigma ~ normal(3, 3); // Normal with mean 3, st.dev. 3

  for(i in 2:N){
    y[i]~ normal(mu+ phi*(y[i-1] - mu), sigma);
  }
}'
```

```r
data <- list(N=N, y=y)
warmup <- 50
niter <- 2000
fit <- stan(model_code=StanModel,data=data, warmup=warmup,iter=niter,chains=4)
```

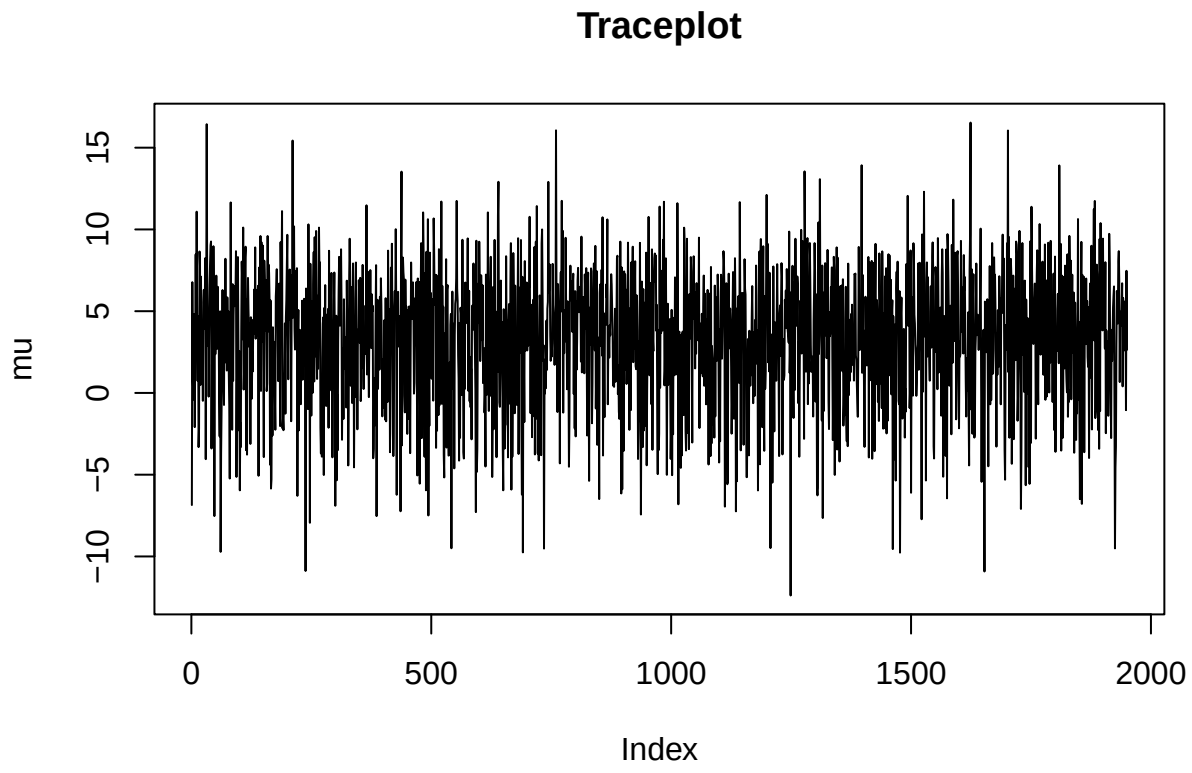
```
## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.

## Trying to compile a simple C file

## Running "C:/PROGRA~1/R/R-44~1.3/bin/x64/Rcmd.exe" SHLIB foo.c
## using C compiler: 'gcc.exe (GCC) 13.3.0'
## gcc  -I"C:/PROGRA~1/R/R-44~1.3/include" -DNDEBUG   -I"C:/Users/Administrator/AppData/Local/R/win-libi
## cc1.exe: warning: command-line option '-std=c++14' is valid for C++/ObjC++ but not for C
## In file included from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/
##                  from C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/
##                  from C:/Users/Administrator/AppData/Local/R/win-library/4.4/StanHeaders/include/stan
##                  from <command-line>:
## C:/Users/Administrator/AppData/Local/R/win-library/4.4/RcppEigen/include/Eigen/src/Core/util/Macros.h
##   679 | #include <cmath>
##       |          ^~~~~~~
## compilation terminated.
## make: *** [C:/PROGRA~1/R/R-44~1.3/etc/x64/Makeconf:289: foo.o] Error 1

## WARNING: Rtools is required to build R packages, but is not currently installed.
##
## Please download and install the appropriate version of Rtools for 4.4.3 from
## https://cran.r-project.org/bin/windows/Rtools/.

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 6.1e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.61 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: WARNING: There aren't enough warmup iterations to fit the
## Chain 1:          three stages of adaptation as currently configured.
## Chain 1:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 1:          the given number of warmup iterations:
## Chain 1:            init_buffer = 7
## Chain 1:            adapt_window = 38
## Chain 1:            term_buffer = 5
## Chain 1:
## Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 1: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 1: Iteration:  450 / 2000 [ 22%]  (Sampling)
```

```
## Chain 1: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 1: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 1: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 1: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 1: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 1: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 1: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1:
## Chain 1:  Elapsed Time: 0.106 seconds (Warm-up)
## Chain 1:                4.248 seconds (Sampling)
## Chain 1:                4.354 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 2.7e-05 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.27 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: WARNING: There aren't enough warmup iterations to fit the
## Chain 2:          three stages of adaptation as currently configured.
## Chain 2:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 2:          the given number of warmup iterations:
## Chain 2:            init_buffer = 7
## Chain 2:            adapt_window = 38
## Chain 2:            term_buffer = 5
## Chain 2:
## Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 2: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 2: Iteration:  450 / 2000 [ 22%]  (Sampling)
## Chain 2: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 2: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 2: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 2: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 2: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 2: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 2: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2:
## Chain 2:  Elapsed Time: 0.064 seconds (Warm-up)
## Chain 2:                4.066 seconds (Sampling)
## Chain 2:                4.13 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 2.8e-05 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.28 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
```

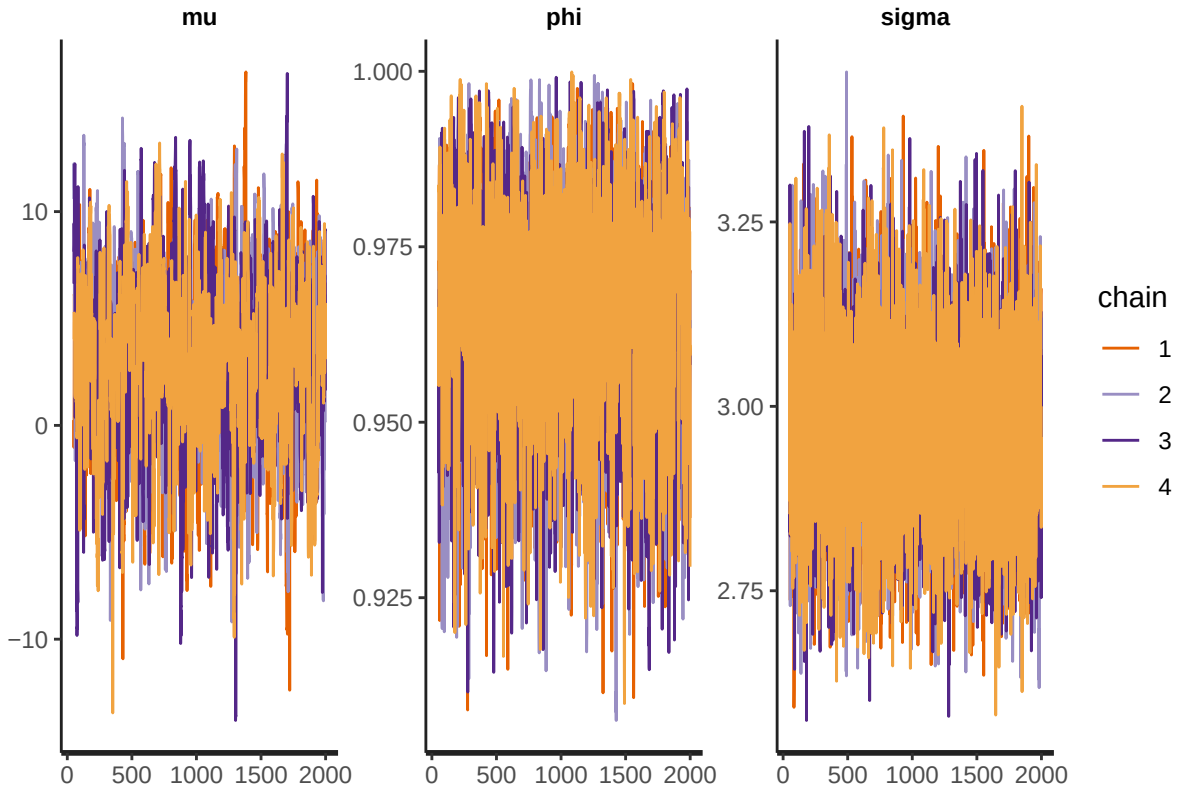
```
## Chain 3: WARNING: There aren't enough warmup iterations to fit the
## Chain 3:          three stages of adaptation as currently configured.
## Chain 3:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 3:          the given number of warmup iterations:
## Chain 3:            init_buffer = 7
## Chain 3:            adapt_window = 38
## Chain 3:            term_buffer = 5
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 3: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 3: Iteration:  450 / 2000 [ 22%]  (Sampling)
## Chain 3: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 3: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 3: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 3: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 3: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 3: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 3: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3:
## Chain 3:  Elapsed Time: 0.101 seconds (Warm-up)
## Chain 3:                4.281 seconds (Sampling)
## Chain 3:                4.382 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 2.7e-05 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.27 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: WARNING: There aren't enough warmup iterations to fit the
## Chain 4:          three stages of adaptation as currently configured.
## Chain 4:          Reducing each adaptation stage to 15%/75%/10% of
## Chain 4:          the given number of warmup iterations:
## Chain 4:            init_buffer = 7
## Chain 4:            adapt_window = 38
## Chain 4:            term_buffer = 5
## Chain 4:
## Chain 4: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4: Iteration:   51 / 2000 [  2%]  (Sampling)
## Chain 4: Iteration:  250 / 2000 [ 12%]  (Sampling)
## Chain 4: Iteration:  450 / 2000 [ 22%]  (Sampling)
## Chain 4: Iteration:  650 / 2000 [ 32%]  (Sampling)
## Chain 4: Iteration:  850 / 2000 [ 42%]  (Sampling)
## Chain 4: Iteration: 1050 / 2000 [ 52%]  (Sampling)
## Chain 4: Iteration: 1250 / 2000 [ 62%]  (Sampling)
## Chain 4: Iteration: 1450 / 2000 [ 72%]  (Sampling)
## Chain 4: Iteration: 1650 / 2000 [ 82%]  (Sampling)
## Chain 4: Iteration: 1850 / 2000 [ 92%]  (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4:
```

```
## Chain 4:  Elapsed Time: 0.051 seconds (Warm-up)
## Chain 4:                3.707 seconds (Sampling)
## Chain 4:                3.758 seconds (Total)
## Chain 4:
```

```r
# Print the fitted model
#print(fit,digits_summary=3)
# Extract posterior samples
postDraws <- extract(fit)
# Do traceplots of the first chain
par(mfrow = c(1,1))
plot(postDraws$mu[1:(niter-warmup)],type="l",ylab="mu",main="Traceplot")
```

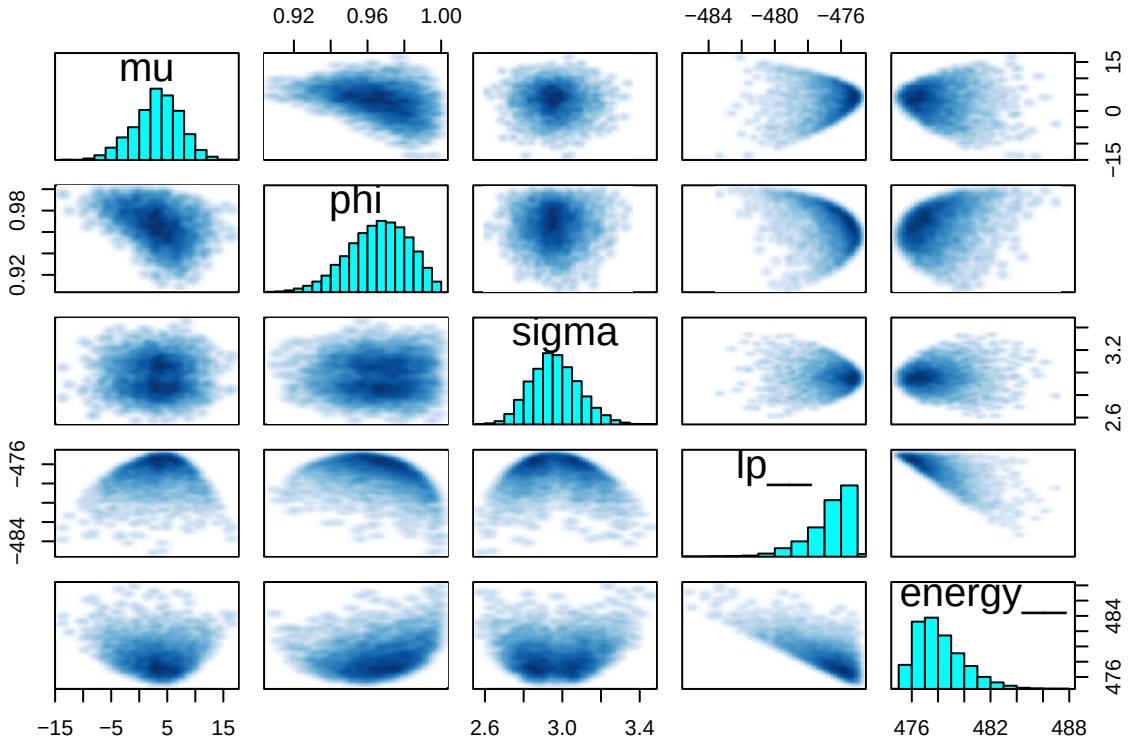



```r
# Do automatic traceplots of all chains
traceplot(fit)
```




```
# Bivariate posterior plots
pairs(fit)
```

```
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
## Warning in par(usr): argument 1 does not name a graphical parameter
```

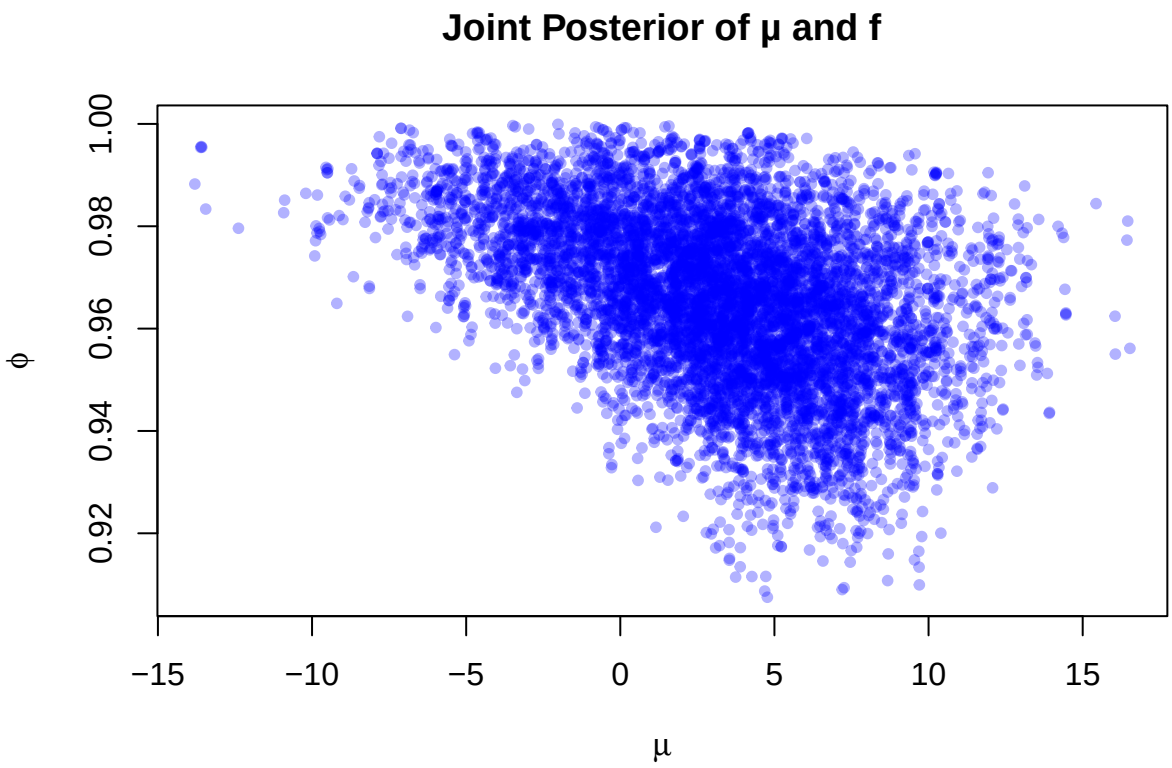



```
summary_fit <- summary(fit, pars = c("mu", "phi", "sigma"))$summary
# extract the mean, 95% CI and effective posterior sample
posterior_summary <- summary_fit[, c("mean", "2.5%", "97.5%", "n_eff", "Rhat")]
print(round(posterior_summary, 3))
```

```
##         mean   2.5%  97.5%    n_eff  Rhat
## mu     3.191 -5.583 10.723  951.075 1.003
## phi    0.966  0.931  0.994 1896.228 1.001
## sigma 2.963  2.739  3.218 7729.057 1.000
```

Mean, 95% CI and effective posterior sample number are shown as above. Despite simulated phi and sigma values are quite close to the true values and the true values lie in the 95% CI, the simulated mean gives too wide 95% CI, making it unable to make a estimate of it. This shows that the data provide little information about , and the estimate of is not very precise.

```
posterior_samples <- as.data.frame(rstan::extract(fit, pars = c("mu", "phi")))

plot(posterior_samples$mu, posterior_samples$phi,
     xlab = expression(mu), ylab = expression(phi),
     main = "Joint Posterior of  and ", pch = 20, col = rgb(0, 0, 1, 0.3))
```



Both cases have Rhat values < 1.05, meaning all chains are well- mixed and agree. Besides, the traceplots have shown no stickiness and no drift. Thus, convergence is considered reached for both ϕ values.
When ϕ is equal to 0.4, μ and ϕ is low correlated (cloud-shape plot) and moderate correlated (diagonal elliptical shape) when ϕ is 0.98. This is due to the near-nonstationary behavior of the AR(1) process when ϕ is close to 1. We can conclude that even if convergence is good or acceptable, it does not necessarily mean a good sample for the posterior.